WORKING PRODUCT SPECIFICATION

COMPETITIVE ADVERTISING ARENA

A live advertising marketplace designed as a spectator game: brands fight for scarce attention, product performance influences competitive efficiency, and temporary promotional bounties may act as a distribution engine.

STATUS

Working thesis, not a frozen specification

Mechanics, economics, legal structure and visual details remain testable.

How to read this document

This brief is intentionally comprehensive, but it is not intended to freeze the product. It records the strongest version of the idea reached in discussion, the reasoning that produced it, the stack currently preferred, and the uncertainties that still deserve testing.

CODING AGENTS: begin with Section 35, Agent execution and handoff playbook. Read Sections 24-33 before changing the visual system, widget runtime, AI model, or security boundaries. Read Section 37 before implementing any spatial / Floor experience. Read Sections 39-42 before implementing launch promotions, referral mechanics, growth instrumentation, or surge-readiness behavior. Read Sections 44-45 before choosing the agent workflow, parallelization strategy, browser/GitHub tooling, review gate, or autonomous-loop configuration. The brief and approved golden references are source-of-truth constraints, not optional inspiration.

> **CORE RULE**
> Treat numbers, mechanics, thresholds and implementation details as hypotheses unless the document explicitly labels them as an external fact. The aim is to preserve thinking, not convert brainstorming into doctrine.

Status language used throughout

Status	Meaning
WORKING THESIS	A strategic belief worth testing; may be central but is still falsifiable.
CURRENT PREFERENCE	Where the discussion currently lands because it appears to offer the best balance.
OPEN PARAMETER	A value or mechanic that should be simulated, tested or tuned rather than assumed.
LAUNCH BLOCKER	A question that should be resolved before public launch or before enabling a specific feature.
LATER POSSIBILITY	A plausible expansion that should not inflate V1.

What is deliberately unresolved

- Working brand: OUTRIVL. Final trademark / naming clearance and any future visual refinements remain open.
- Exact spending caps for Indie, Startup and Open.
- Attack price escalation and decay equations.
- The exact quantity of guaranteed inventory after a successful attack.
- The formula, inputs and allowed range of Audience Score.
- The legally permissible structure of temporary cash promotions in each jurisdiction.
- The final payment provider(s), because approval may depend on the promotion design.
- The exact revenue share offered to publishers if a distributed network is built.

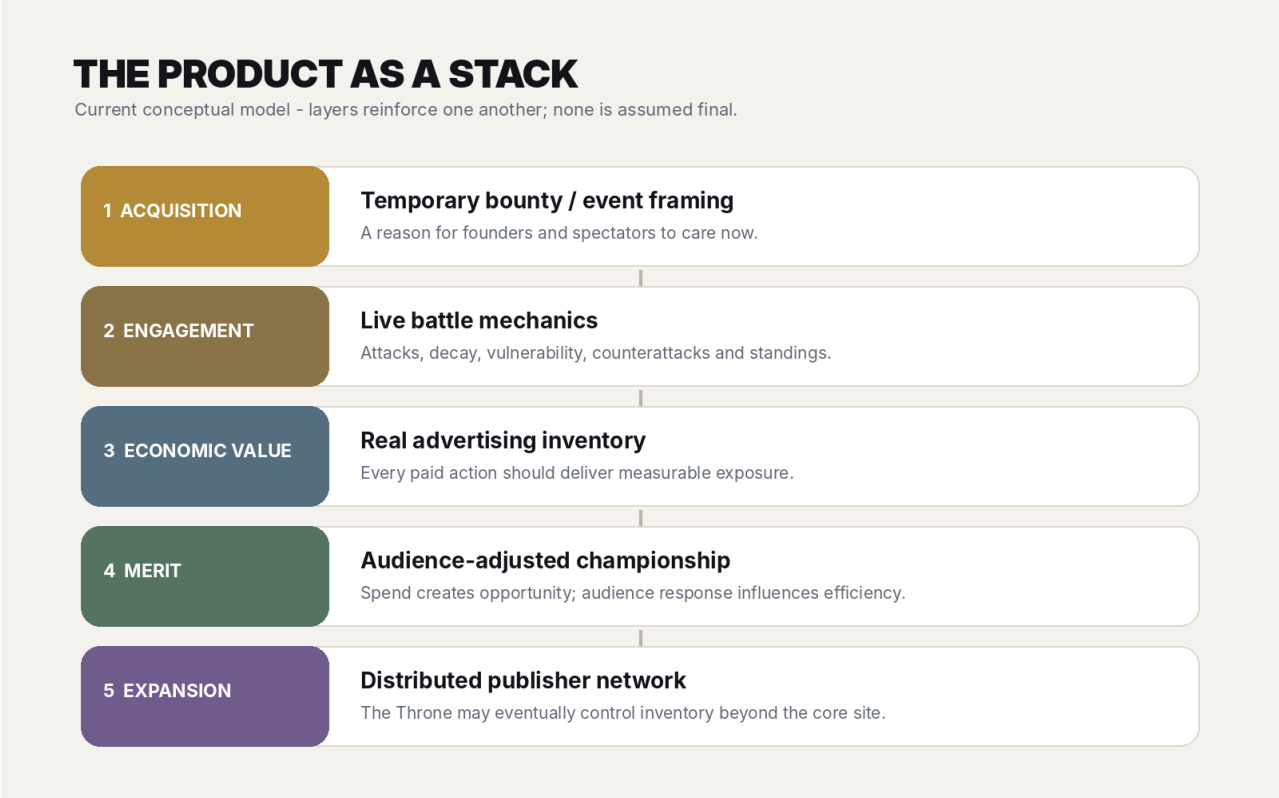
Contents

1. Executive summary

WORKING THESIS

Build a live multiplayer advertising marketplace where companies compete publicly for scarce advertising inventory. The auction becomes visible, strategic and entertaining; the products remain the protagonists.

The weak version of the idea is a paid leaderboard: whoever spends the most sits at the top. That primitive already exists in multiple forms and does not appear sufficient on its own. The stronger version turns the advertising market itself into a spectator product while preserving genuine advertiser value.



Conceptual stack - acquisition, engagement, economic value, merit and eventual network expansion.

The strongest version reached in discussion

- Three competitive classes are contemplated: Indie, Startup and Open. They are spending / weight classes, not industry arenas.
- Industry labels such as AI, DevTools, Productivity, Mobile and Consumer are filters and metadata, preserving liquidity rather than fragmenting it.
- The current King controls premium advertising inventory. A seasonal Champion is scored separately so spend alone need not decide the title.
- Money functions as ammunition: it creates the ability to attack and acquire exposure, but the championship can incorporate bounded audience-performance effects.
- Attack prices rise after takeovers and may decay during quiet reigns, creating vulnerability and timing strategy.
- Every paid attack should buy real advertising value, potentially through guaranteed impressions, a short shield, or a hybrid.
- Audience quality should be measured from an independent randomized evaluation stream so rich advertisers do not automatically obtain stronger quality scores by buying more exposure.
- Temporary cash bounties remain a central distribution hypothesis. They may unlock only after revenue milestones so the event earns the right to become expensive.

- The promotion is not assumed to be legally safe. Its structure is a launch blocker requiring specialist review and payment-processor approval.
- Long term, the Throne could control inventory distributed across partner websites, newsletters and apps, turning the concept into a category-aware ad network rather than a single-site novelty.

Why this might be unusually attractive

The initial software is compact relative to the possible upside. The difficult problems are not model training or native application development; they are liquidity, advertiser ROI, adversarial traffic, game balance, visual quality and the legal treatment of promotions. That makes the concept suitable for a fast but serious experiment.

> **MOST IMPORTANT BEHAVIOURAL TEST**
> When an advertiser is dethroned, do they voluntarily spend again to reclaim the position? Re-attack rate is a much stronger validation signal than compliments, launch traffic or social likes.

2. How the idea evolved

Stage A - paid King of the Hill

The starting concept was a live advertising leaderboard in which companies, communities or projects bid to control a dominant top slot. A visible jackpot funded by a percentage of bids would grow with activity, and whoever held the top spot at a deadline would win.

The immediate strengths were obvious: rivalry, public status, FOMO, a live number to watch and a reason for communities to mobilize. The weakness was equally important: in the naive version, the participant with the most money simply wins.

Stage B - separate advertising from merit

The design then shifted from money = score to money = ammunition. Spending would still matter because the platform sells advertising, but daily or seasonal caps could stop a whale from ending protected competitions. Timing, price decay and audience response could influence competitive efficiency.

Stage C - King and Champion become different

Separating the immediate owner of premium inventory (King) from the season winner (Champion) created a much more useful narrative. A rich advertiser may be able to seize inventory frequently, while a smaller advertiser can potentially win a season through timing, efficiency and stronger audience response.

Stage D - the bounty becomes distribution, not the permanent economy

The promotional prize initially looked like a permanent jackpot. That creates serious legal and processor concerns. The stronger framing is a temporary acquisition event: a bounty may attract founders and spectators who would ignore another ad platform, while the permanent product must remain worthwhile as advertising without cash.

Stage E - do not fragment by vertical

A later drift toward many industry arenas was rejected. Competitive marketplaces need density. The current preference is three spending classes with product-category filters, summarized as: three economies, many views.

Stage F - the long-term company becomes a network

The most important expansion insight was that the Throne could eventually control advertising inventory across external publisher surfaces. At that point every embedded ad can both serve the current King and invite challengers, giving the marketplace a potential self-distribution mechanism.

3. Product thesis and market roles

The product has to satisfy three constituencies at once. If any one of them receives no value, the loop breaks.

Advertisers

- Need measurable impressions / visits and eventually conversions.
- Need a rational reason to attack beyond novelty.
- Need transparent rules and confidence that the algorithm will not arbitrarily erase the value of spend.
- May value prestige, rivalry, records and shareable wins as additional benefits.

Spectators / discoverers

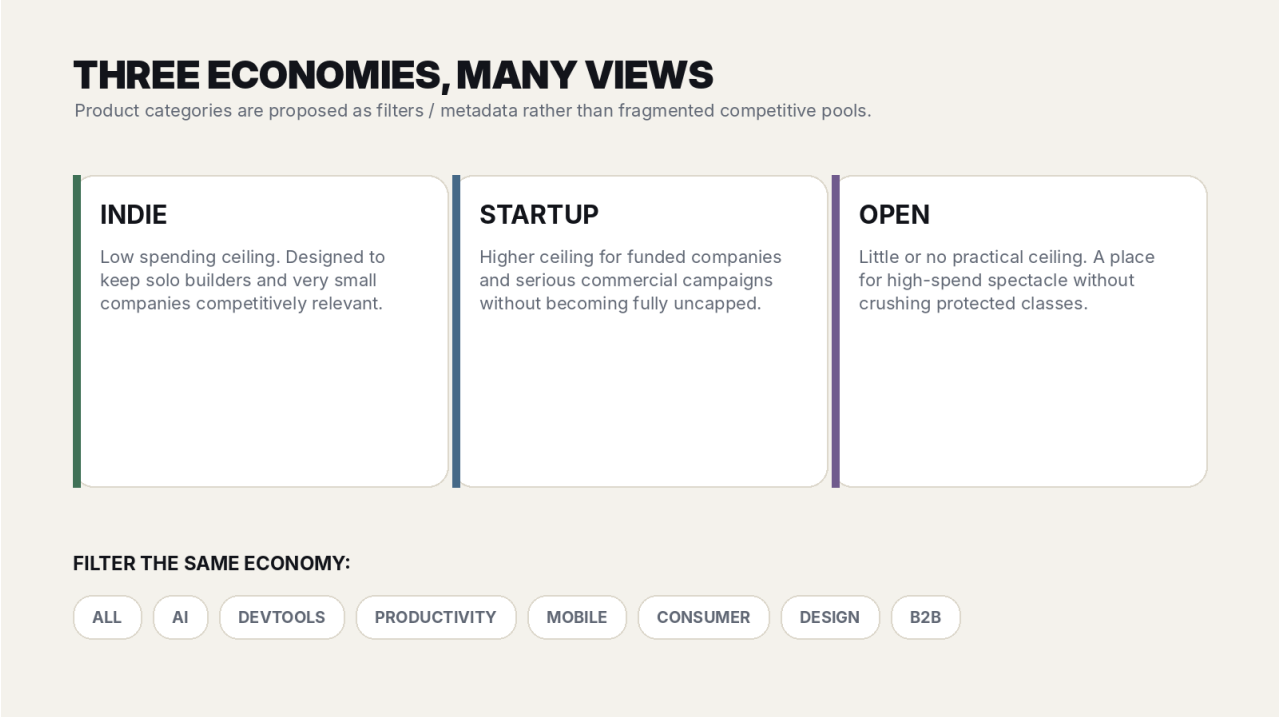
- Need interesting products, not rows of financial numbers.
- Need movement: dethronements, vulnerability, close standings, comebacks and records.
- Should be able to browse without creating an account.
- May visit primarily for product discovery, competition, bounty progress or social drama.

Publishers - later

- Would supply external inventory.
- Need easy integration and a revenue share competitive with alternatives.
- May benefit from a unit that is also content: who currently rules the relevant class / category view?

> **DESIGN PRINCIPLE**
> A boring product should not become interesting merely because it paid. A genuinely interesting product should become more interesting because the audience can watch it fight.

4. Three competitive classes and category filters



Current structural preference - class determines competitive spending rules; category determines discovery views.

Indie

Intended for solo builders, bootstrappers and very small teams. A deliberately low ceiling would protect the battlefield from funded entrants. Exact limits remain an open parameter; early discussion used figures such as $25-$50 per day as examples, not commitments.

Startup

A higher-budget class for funded startups and more serious commercial campaigns. It may use the same game primitives but with a materially higher cap and perhaps different inventory economics.

Open

A class with little or no practical spending ceiling. If two large advertisers want to create an absurd public bidding war, Open gives them somewhere to do it without destroying the protected classes.

Why categories are filters

Category fragmentation would create many thin markets. Instead, a spectator might view Indie → AI or Startup → DevTools while the underlying class remains unified. Filters can also support sorting by Championship, Trending, Audience, Efficiency or Newest without creating a new economic pool.

> **OPEN PARAMETER**
> Whether filters affect only browsing or also influence which randomized evaluation impressions an advertiser receives. Relevance may improve signal quality, but too much segmentation could recreate the liquidity problem indirectly.

5. Core game economy

Money as ammunition

The economic premise is intentionally not egalitarian: advertising spend must matter. But protected classes can make bankroll a bounded resource rather than an unlimited win condition. Spending creates attack opportunities and real exposure; championship scoring can reward efficient use of that opportunity.

Attack price

The Throne exposes a current attack price. A challenger spends the required amount to seize the premium position. The successful attack may raise the next required price. The escalation rule is an open parameter and should be simulated rather than guessed.

Price decay / vulnerability

During an uninterrupted reign, the attack price may gradually fall. This makes long-held positions increasingly vulnerable and gives spectators a readable state change: the King is becoming cheaper to overthrow. The decay can be calculated deterministically from the takeover price and reign start time rather than requiring constant database writes.

Guaranteed value after attack

A pure instantaneous auction has a bad failure mode: an advertiser pays, is dethroned seconds later and receives almost nothing. Several remedies are possible:

- Guaranteed minimum premium impressions.
- A brief takeover shield / immunity period.
- A hybrid where a short shield is followed by vulnerability only after a minimum delivery threshold.

Traffic allocation

A working concept discussed was to give the King the majority of premium inventory, retain some inventory for active challengers, and reserve a meaningful independent pool for quality evaluation. Exact percentages are open parameters. The guiding rule is that holding the Throne must be measurably valuable while the evaluation stream remains statistically useful.

Daily / seasonal caps

Caps can be enforced transactionally against the authoritative ledger. They should be designed so a user cannot bypass them through simultaneous requests, multiple sessions or race conditions. The competitive question is not whether money matters, but how much marginal advantage additional spend should purchase before a class stops feeling contestable.

6. Audience quality and championship scoring

Why raw CTR is not enough

CTR can reward clickbait. Community votes can reward brigading. Purchased traffic can create circular advantage. The aim is not to declare one product objectively better; it is to estimate how effectively a product earns the attention of a comparable audience when given an equal opportunity.

Randomized evaluation stream

Every active advertiser could receive a comparable sample of randomized evaluation impressions independent of spend. Performance on that sample would generate an Audience Score or similar metric. This prevents a rich advertiser from acquiring a better quality score simply because it already bought the most exposure.

Possible signal inputs - later weighting required

- Unique click-through rate.
- Bot-adjusted and self-click-adjusted engagement.
- Landing-page engagement / qualified visit measures where privacy and implementation allow.
- Repeat interest or return behaviour.
- Optional downstream conversion events for advertisers willing to integrate them.
- Limited audience preference signals, only if resistant to coordinated manipulation.

Bounded multiplier

The discussion repeatedly converged on a deliberately modest multiplier, with illustrative ranges around 0.85x-1.15x or 0.80x-1.20x. The point is philosophical rather than numeric: quality should create an edge, not turn paid inventory into an inscrutable algorithmic lottery.

Crown Points

The simplest conceptual model is throne time multiplied by a bounded Audience Score. Other versions could incorporate attack efficiency, guaranteed-delivery completion, or diminishing returns. The scoring model should be understandable enough that advertisers can reason about it and spectators can follow the race.

Efficiency metrics

- Crown Points per dollar.
- Throne minutes per dollar.
- Qualified visits per dollar.
- Dethronements / successful counterattacks.
- Spend rank versus championship rank.

> **HEALTH SIGNAL**
> If final championship rank becomes almost identical to spend rank, the game is not rewarding strategy / audience response enough. If spend barely matters, the advertising business loses its economic reason to exist.

7. Seasons, prestige and discovery

Season resets

A weekly season was the strongest early hypothesis because it creates urgency, repeated fresh starts and understandable history. Seven days is not fixed. The important principle is periodic reset: permanent incumbency kills hope, while persistent historical records preserve prestige.

Persistent records

- Championship count.
- Lifetime Throne time.

- Longest reign.
- Total dethronements.
- Successful counterattacks.
- Highest Audience Score.
- Best Crown efficiency.
- Biggest comeback.
- Lifetime visits generated.
- Lifetime advertising spend.

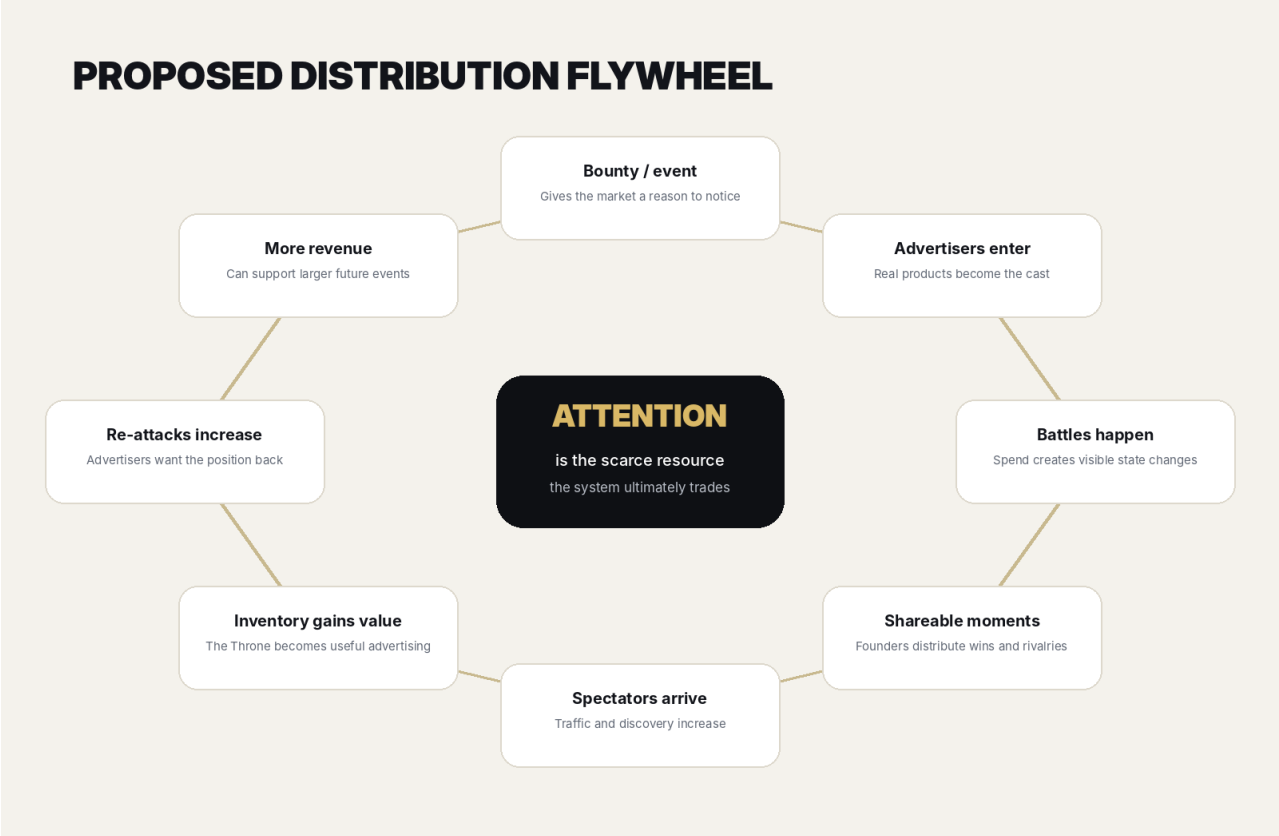
Product profile as durable identity

Unlike a one-day launch feed, a product can develop a competitive record. If the network becomes meaningful, badges such as 3x Indie Champion or Best Audience Score may become useful social proof rather than novelty decoration.

Product Hunt-like discovery, but persistent and live

The product card should lead with logo, product name, excellent one-line pitch and strong imagery. Game metrics should enhance that presentation, not replace it. A visitor should be able to discover a compelling product even if they do not understand the game yet.

8. Promotional bounty as distribution



The bounty is treated as an acquisition hypothesis that should strengthen - not substitute for - advertising value.

Why the promotion stays central

A founder may ignore another advertising marketplace. A public event with free money, a deadline and visible competition is much harder to ignore. The promotion can therefore perform the job that paid acquisition normally would: attract the first cast of advertisers and give spectators a reason to watch.

Self-funding unlock model

Instead of promising a large prize before knowing whether the event works, the prize can unlock at predefined verified-revenue thresholds. Illustrative examples discussed included $50 or $100 at the floor, then larger tiers such as $250, $500, $1,000 or $2,500 only after materially larger event revenue. The exact ladder must be calculated after fees, taxes, fraud risk, refunds, publisher payouts and desired margin.

The unlock meter is itself content

A visible progress bar creates another live state variable. Participants may have an incentive to share the event because more activity can unlock a larger prize. This could make the bounty more than a giveaway: it can turn participants into distributors of the marketplace.

Do not teach the market that cash is the only reason to return

The permanent product must stand on real advertising economics. Cash is best thought of as ignition, seasonal amplification or sponsor-funded spectacle. If the marketplace dies whenever the bounty disappears, the core product has not been validated.

Alternative non-cash prizes

- Exclusive homepage / network takeover.
- Bonus premium impressions.
- Advertising credits.
- Sponsor-provided credits / services.
- Featured placement in the following season.

> **CRITICAL DISTINCTION**
> Self-funding reduces financial exposure. It does not automatically make the promotion legal. Commercial design and legal design are separate workstreams.

9. Legal and payment-processor constraints

> **LAUNCH BLOCKER**
> Cash-prize promotion rules must be reviewed by specialist counsel in every intended operating / participant jurisdiction before launch. This document deliberately does not claim that any proposed structure is legal.

India - why the risk is material

India's Promotion and Regulation of Online Gaming Act, 2025 defines an online money game broadly: an online game, whether based on skill, chance or both, played by paying fees, depositing money or other stakes in expectation of monetary or other enrichment. The Act also includes companies within the definition of person, and it prohibits offering online money games, advertising them and facilitating related transfers. The 2026 rules came into force on 1 May 2026.

That does not automatically resolve whether this product is an online game at all, rather than an advertising marketplace with a promotion. That classification question is exactly why specialist advice matters. The safe design principle is to ensure that paid transactions buy genuine advertising inventory and to keep the permanent business economically coherent without prize money.

Possible structural direction - not legal advice

One concept discussed was to separate paid advertising from promotion qualification: every entrant could receive an equal free evaluation allocation, while paid purchases buy additional advertising exposure but do not directly purchase the cash-winning score. Whether that is sufficient, desirable or necessary is unresolved and must be assessed by counsel.

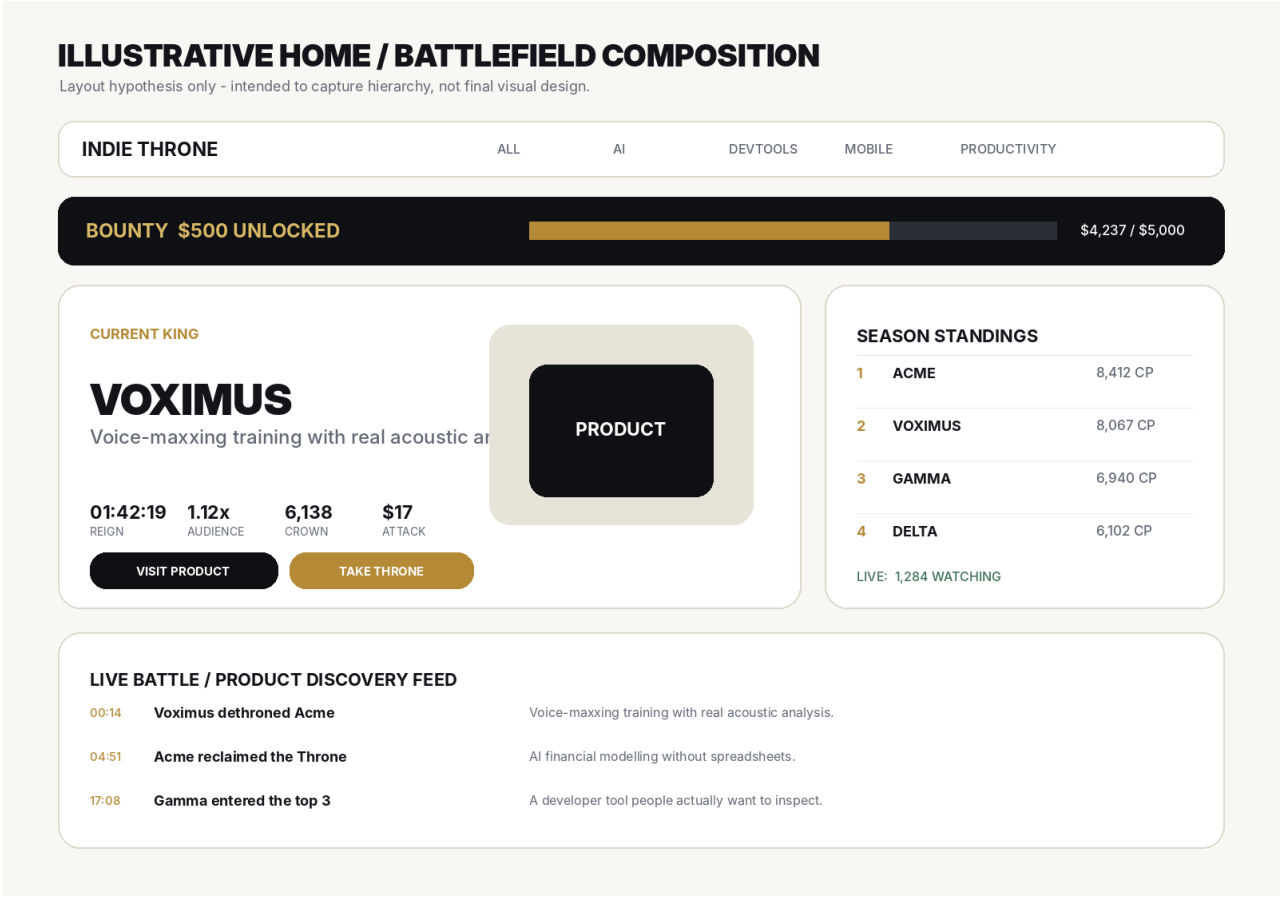
Payment processors may be stricter than law

Stripe's published restricted-business guidance explicitly says its gambling policy can include skill or chance games with cash prizes, sweepstakes and pay-in auctions. Even a promotion that counsel considers legal may still require processor approval or a different processor. Therefore the payment abstraction should not make the product depend on a single provider.

Operational implications

- Do not market the permanent product as gambling. The economic transaction is advertising inventory.
- Do not rely on a checkbox to prevent chargebacks. Maintain evidence of delivery, clear terms, advertiser verification and an auditable ledger.
- Geofencing or excluding jurisdictions may become necessary for promotional events.
- Formal contest rules, eligibility, payout verification and sponsor terms may be required.
- The payment provider should review the actual final business model, not a sanitized one-line description.

10. Spectator UX and headline system



Illustrative information hierarchy. The final visual direction should be substantially more art-directed than this wireframe.

Visual thesis

The strongest description reached in discussion is Product Hunt crossed with a live sports broadcast - premium editorial product presentation surrounded by game-state metrics. It should not read as a casino, crypto dashboard, generic SaaS admin panel or Bloomberg terminal.

The product remains the hero

A listing should require a strong logo, product name, excellent one-line headline, short description, hero image or product visual, CTA and category metadata. The current King receives a disproportionately large, beautiful presentation rather than merely a first row in a table.

Curated spectator metrics

- Reign duration.
- Crown Points.
- Audience Score.
- Dethronements.
- Efficiency.
- Visits generated.
- Attack price / vulnerability.
- Bounty progress during promotional events.

Headline examples

- Voximus takes the Indie Throne.
- Acme ends a 4-hour reign.
- Tiny startup reaches #2 spending only $18.
- Audience favourite closes to within 210 Crown Points.
- Bounty doubles to $500.
- Smallest spender now leads the season.

Motion philosophy

Most of the interface should be restrained and premium. Motion becomes theatrical only when state changes matter: dethronement, bounty unlock, major rank change or season close. If everything moves, nothing feels important.

11. Long-term distributed publisher network

The key expansion hypothesis

The Throne eventually controls inventory not only on the core site but across embedded publisher units. A niche AI site, newsletter, app or directory might display the current relevant King and offer a path for challengers to enter the battle.

Why this changes the company

At that point the advertiser is no longer paying for attention on a quirky leaderboard. They are buying category-relevant exposure across a network. Publishers receive revenue share; advertisers receive inventory; the platform takes margin; spectators continue to create narrative around the auction.

Self-distributing ad unit

A conventional ad says Try Product X. A live arena unit can say Product X currently holds the Indie Throne - enter the battle. Every ad impression can therefore also advertise the marketplace itself, potentially recruiting the next challenger.

Network-effect thesis

More publishers → more inventory → better advertiser value → more advertiser demand → more competition → higher revenue → stronger publisher payouts → more publishers. This is a later-stage hypothesis, not a reason to overbuild V1.

12. Business model and economics

Primary revenue

Advertising spend is the core. The platform should retain a margin after payment fees and, later, publisher payouts. The precise revenue share is unresolved.

Additional revenue possibilities

- Sponsored seasons / championships.
- Sponsored bounty contributions.
- Premium advertiser analytics and attribution.
- Automated campaign / attack strategies later.
- Multi-class campaign management.
- API access and advanced integrations.
- OUTRIVL Pro / Cloud subscriptions for managed AI creative tooling, widget generation, richer analytics, collaboration and convenience; BYOK remains a first-class alternative.
- Advanced first-party intent / performance analytics derived from clearly defined listing and widget interactions, subject to privacy, aggregation and consent rules.
- Interactive widget hosting / distribution and, later, publisher-network inventory with a platform margin.

Prepaid credits

Rather than processing a new card transaction for every attack, advertisers can preload credits and spend them instantly. This reduces fixed payment-fee drag and makes rapid counterattacks possible. The authoritative balance should be backed by an append-only financial ledger, not a single mutable number.

Promotion economics

The prize ladder should be modeled as acquisition spend with guardrails. The preferred launch implementation is a guaranteed funded floor plus discrete public unlock tiers, not an uncapped percentage of volume. Each additional tier should be financially reserved before it is announced as unlocked, and the event should have a hard maximum pool. The exact volume trigger - verified marketplace revenue, qualified activity, sponsor contributions, or another objective measure - requires promotions counsel and processor review. Participant purchases must not directly improve promotional score, eligibility or chance of winning. Prize growth should never outrun the unit economics simply because the UI makes a larger number exciting.

13. Fraud, moderation and adversarial design

Because audience response affects competitive strength and money changes hands, abuse should be assumed rather than treated as an edge case.

Threats to anticipate

- Self-clicking and bot traffic.
- Coordinated brigading.
- Multiple-account cap evasion.
- Payment fraud and chargebacks.
- Malicious redirects / unsafe landing pages.
- Fake advertiser identities.
- Attempted manipulation of randomized evaluation traffic.
- Race conditions on attacks and spending caps.
- Collusion between advertisers.
- Artificial viewer-count inflation.

Controls

- Advertiser verification and domain ownership checks.
- URL / creative moderation.
- Cloudflare Turnstile at sensitive flows.
- Redis-backed rate limits and short-lived dedupe keys.
- Server-side / Postgres authoritative scoring and ledger rules.
- Bot-adjusted engagement metrics.
- Evaluation traffic segregation and integrity monitoring.
- Append-only audit events.
- Clear prohibited-category policy.

Why crypto is not preferred for launch

Crypto could generate tribal bidding wars, but it also imports disproportionate scams, botting, payment risk, regulation and moderation. The first market should probably be cleaner software products, while category metadata remains flexible.

14. Launch / V1

> **CURRENT PREFERENCE**
> Launch Indie, Startup and Open from day one. Each class has its own Throne, standings, price state, caps / rules and season history. Seed enough real products to make at least one class visibly active; marketing may emphasize the class that reaches liquidity fastest, but the other classes remain usable and visible. Run one short season and prove the battle loop before adding publisher inventory or elaborate scoring.

V1 capabilities

- Advertiser onboarding and verification.
- Prepaid credits.
- Current Throne state and transactional attack endpoint.
- Attack price escalation and deterministic decay.
- Spending cap enforcement.
- Guaranteed inventory / takeover protection mechanism.
- Randomized evaluation allocation.
- Simple bounded Audience Score.
- Crown Points and season standings.
- Category filters / sorting.
- Battle feed and share cards.
- Impression and click tracking.

- Advertiser analytics.
- Bounty display / unlock logic only if legally and processor-approved.
- A bounded launch-pool meter with authoritative server-side unlock state, hard cap and public rules if the promotional structure is legally and processor-approved.
- Founding Pro grants and a verified referral-reward system that rewards product benefits / Cloud credits without changing competitive rank, Crown Points, promotional score or prize eligibility.
- Feature flags / surge controls that can disable the Floor, public presence, AI generation, nonessential realtime or referral processing independently while preserving product pages, read traffic and transactional economic correctness.
- Basic moderation, bot protection and abuse controls.
- Indie, Startup and Open are all visible and usable from launch; each has its own Throne, standings, price state, caps / rules, season records and history.
- Permanent product listing pages with OUTRIVL-specific historical performance records.
- Interactive product widgets as a first-class listing primitive, not a later bolt-on.
- Responsive widget states tied to context / available canvas: ROW → CARD → FEATURE → THRONE, plus FLOOR_BOOTH and PRODUCT_PAGE surfaces.
- A constrained OUTRIVL Widget Kit and server-side schema validation; no arbitrary advertiser JavaScript in the main page.
- Widget interaction telemetry: starts, completions, dwell, interaction rate, downstream click-through and other clearly defined first-party events.

Not required for the first complete release

- A staged unlock for Startup is not part of the current plan; Startup is live from day one with its own Throne, standings and economics.
- A staged unlock for Open is not part of the current plan; Open is live from day one with its own Throne, standings and economics.
- Distributed publisher marketplace.
- Native mobile app.
- Complex AI quality judging.
- Crypto vertical.
- Sophisticated conversion attribution.
- Autonomous bidding agents.
- Separate industry arenas.

Illustrative starting values - all open parameters

Parameter	Illustrative discussion range / concept	Why unresolved
Season length	~7 days	Need enough urgency without forcing constant participation.
Indie cap	$25-$50/day examples	Needs simulation against attack price and expected inventory value.
Evaluation inventory	~20% example	Must balance statistical quality against King value.
Audience multiplier	~0.85x-1.15x example	Must matter without making spend feel arbitrary.
Seed advertisers	~10	Enough density for visible conflict without recruitment burden.

15. Metrics, experiments and kill criteria

Primary validation: re-attack rate

The strongest behavioural signal is whether a dethroned advertiser comes back and spends again. It captures both game motivation and perceived advertising value better than vanity traffic.

Advertiser metrics

- First-purchase conversion.
- Re-attack rate.
- Spend per advertiser.
- Share hitting cap.
- Retention across seasons.
- CPC / qualified visits.
- Conversion where available.
- Crown Points per dollar.

Game-health metrics

- Throne changes per day.
- Median and distribution of reign length.
- Unique attackers.
- Comeback frequency.
- Standings closeness.
- Correlation between spend rank and championship rank.
- Concentration of Crown Points.

Spectator metrics

- Return rate.
- Product-card CTR.
- Battle-feed interaction.
- Bounty-meter engagement.
- Share-card referral traffic.
- Spectator-to-advertiser conversion.

Bounty-specific metrics

- Advertisers acquired during promotional periods.
- Prize cost per acquired advertiser.
- Unlock velocity.
- Participant referrals.
- Incremental re-attack rate during bounty periods.
- Prize as a percentage of contribution margin.

Failure / kill signals

- Advertisers buy once for novelty and rarely re-attack.
- The Throne does not deliver traffic valuable enough to rationalize spend.
- Championship rank is effectively identical to spend rank despite quality mechanics.
- The audience only watches money move and does not click / discover products.
- Fraud costs or moderation demands dominate economics.
- The only viable acquisition path is a cash prize larger than the business can sustainably support.

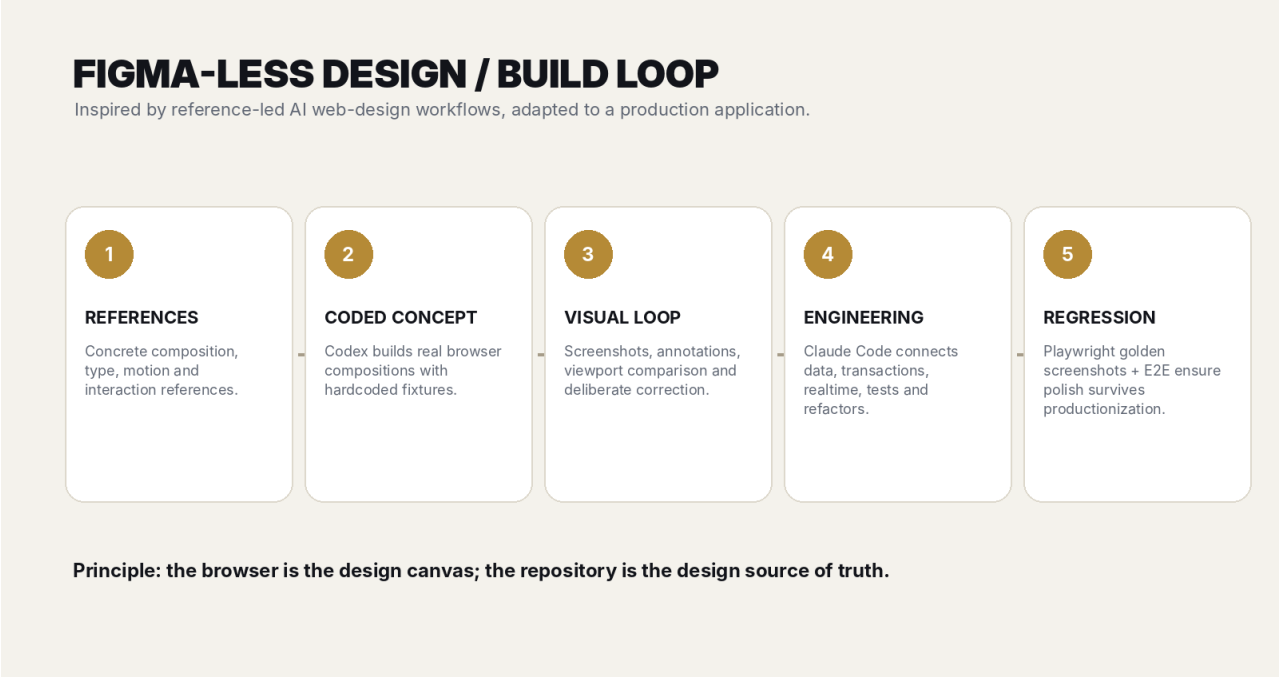
16. Visual direction and Figma-less workflow

Quality bar

The site should be visually compelling enough that screenshots of the interface function as marketing material. The reference point is not a specific copied aesthetic; it is the level of deliberate art direction seen in high-end, motion-rich web work such as Viktor Oddy's recent AI-assisted website demonstrations.

Do not build a generic component-library dashboard

- Avoid default shadcn-style KPI grids and repeated bordered cards as the dominant visual language.
- Products need enough visual real estate to look desirable.
- Metrics should feel editorial / broadcast-like, not like admin analytics.
- Motion should explain state change and create drama, not decorate every scroll.
- The current King should visually dominate the composition.



Proposed production loop - browser-first and reference-led, with no mandatory Figma handoff.

Why Figma can be omitted

For a solo / AI-assisted build, a separate mockup-to-implementation handoff may create unnecessary translation work. The browser can be the design canvas: hardcoded fixtures first, then screenshot comparison, responsive refinement and only then real data integration.

Codex and Claude Code roles

A useful division of labour discussed is to let Codex act as the visual design engineer - generating coded directions, iterating against references and polishing motion - while Claude Code productionizes the selected direction, implements system logic, refactors and tests. The agents should not simultaneously redesign the same component tree.

Repository design constitution

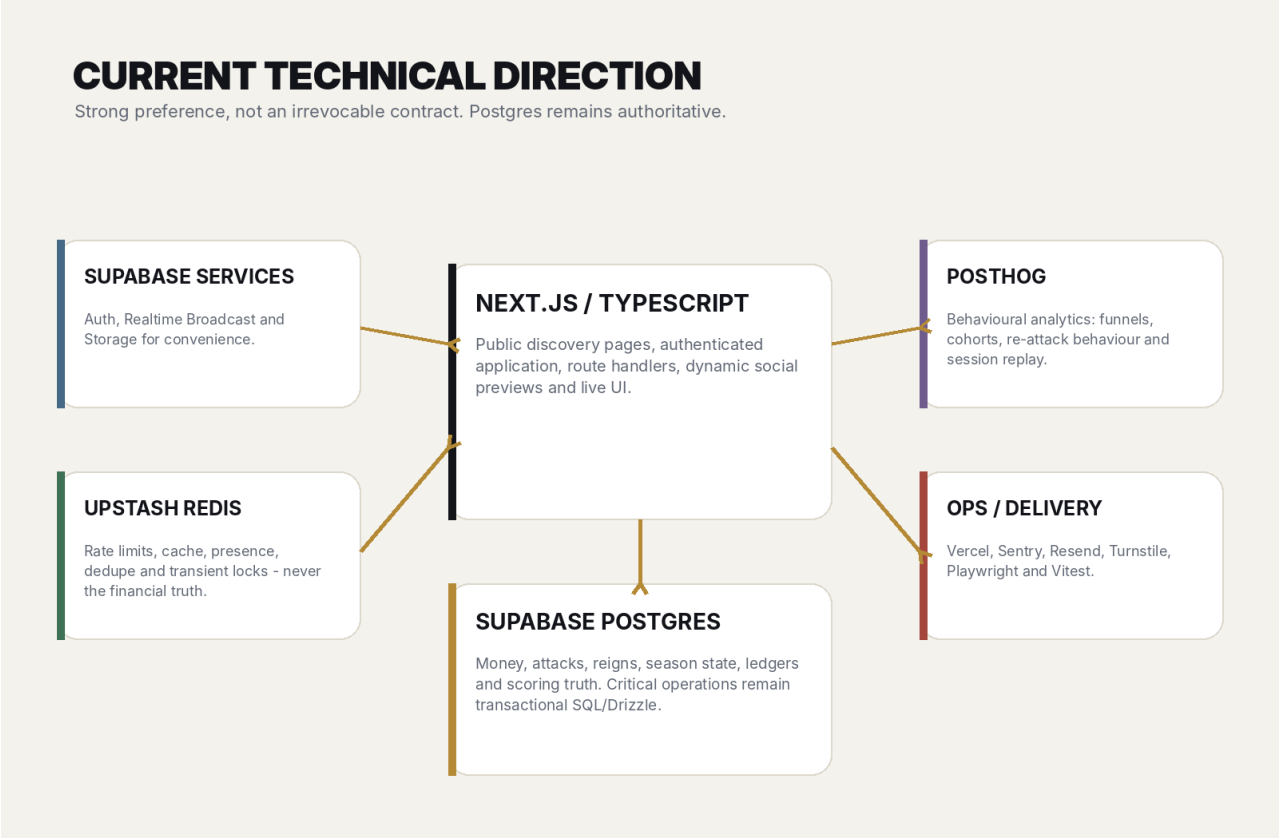
- Never introduce a conventional SaaS-dashboard aesthetic by default.
- Do not turn important composition areas into generic cards without a visual reason.
- Products remain the protagonists.

- Category tags remain filters, not new competitive economies.
- Do not visually drift from approved golden screenshots without explicit reason.
- Use Playwright screenshots as visual regression artifacts.

17. Technology stack decision

> **CURRENT PREFERENCE**
> Next.js + React + TypeScript; Tailwind plus bespoke CSS; Motion with selective GSAP; Supabase Postgres/Auth/Realtime/Storage; Drizzle; Upstash Redis; PostHog; Sentry; Resend; Turnstile; Playwright + Vitest; Vercel initially.

This stack was chosen after explicitly comparing simpler React/Vite/Hono approaches, plain Postgres versus Supabase, self-hosted Redis versus Upstash, Gmail SMTP versus Resend, and Vercel versus a VPS. It is a convenience / portability compromise rather than a belief that every service is technically necessary.



Current architecture direction. The core rule is Postgres truth, Redis speed, Realtime presentation.

Next.js - why it won narrowly over Vite + React + Hono

A Vite/Hono split has a cleaner mental model and strong community support, especially for app-heavy products. However, this product is simultaneously a public product-discovery site, an indexable catalogue, a social object with dynamic share cards, a realtime game and an authenticated advertiser app. Next.js bundles SSR/SSG, routing, metadata, dynamic Open Graph images and route handlers in a way that reduces integration work.

The implementation should still avoid framework theatre: no unnecessary Server Actions, complex caching, edge-only logic or Next-specific magic without a concrete reason.

Supabase - why use it instead of plain managed Postgres

Supabase is not required. The reason to prefer it is that it provides real Postgres plus Auth, Realtime, Storage, a management dashboard and production backup options in one coherent system. Critical game logic should still use ordinary SQL / Drizzle and remain portable. Supabase is infrastructure convenience, not the business architecture.

Upstash Redis - why include it from day one

The product is realtime, adversarial and potentially bursty. Redis is useful for rate limiting, presence, deduplication, short-lived locks and cached public state. Upstash removes Redis server administration and fits Vercel/serverless deployment. Redis never becomes the only record of money, attacks or winners.

PostHog - why it stays

Postgres records what economically happened. PostHog helps explain how humans behaved: funnels, return behaviour, re-attack paths, bounty interactions and session replay. Rebuilding those analytics in the core database would be wasted effort.

Resend - why not Gmail SMTP

Gmail SMTP can send mail, but a transactional service removes deliverability, bounce, logging and operational friction. Resend is inexpensive / free at low volume and supports domain-authenticated transactional email. Convenience is worth more here than the savings from running through a personal mailbox.

Vercel - why use it despite having a VPS

Vercel is not required to run Next.js. The value is deployment convenience, previews, rollback, CDN/WAF and the ability to evaluate multiple Codex/Claude branches quickly. The app should avoid proprietary infrastructure dependencies so hosting can move later if economics or scale make Vercel unattractive.

18. Architecture and data integrity

Source of truth rule

> **NON-NEGOTIABLE ENGINEERING PRINCIPLE**
>
> Postgres is authoritative for credits, payments, attacks, Throne ownership, reigns, spending caps, Crown Points, season results and bounty state. Redis and Realtime may cache or announce state; they do not decide it.

Transactional attack flow

1. Receive an authenticated attack request and an idempotency key.
2. Use Redis / middleware to reject obvious spam or duplicate in-flight requests.
3. Begin a Postgres transaction and lock the relevant Throne state row.
4. Recalculate the authoritative current attack price using server time.
5. Verify credit balance and class spending cap.
6. Debit the append-only ledger.
7. End the previous reign, begin the new reign and record the attack event.
8. Update authoritative Throne state and any guaranteed-inventory commitment.
9. Commit.
10. Broadcast the committed state via Supabase Realtime and update caches.

Append-only credit ledger

Avoid treating balance as a mutable number with no history. Every top-up, attack, promotional credit, refund or adjustment should create a ledger entry with references and idempotency keys. The current balance can be derived or maintained as a consistent projection, but the ledger preserves auditability.

Redis responsibilities

- Attack / endpoint rate limiting.
- IP and device throttles.
- Short-lived idempotency / in-flight locks.
- Presence / active-viewer estimates.
- Short TTL leaderboard / Throne cache.
- Click and impression dedupe windows.
- Fraud heuristic counters.

Realtime responsibilities

- Dethronement events.
- Standings changes.
- Bounty milestone updates.
- Live battle feed.
- Possibly spectator presence approximations.

Suggested domain tables - conceptual

Area	Possible tables
Identity	users, organizations, organization_members
Products	products, product_categories, product_assets
Competition	classes, seasons, season_entries, throne_state, attacks, reigns, battle_events
Money	wallets, wallet_ledger, payment_transactions, refunds
Delivery	impressions, clicks, guaranteed_inventory
Quality	evaluation_impressions, audience_scores, score_snapshots
Promotion	bounties, bounty_tiers, eligibility / verification records

19. Infrastructure / cost snapshot

Pricing below is a research snapshot as of 8 August 2026, not a forecast. Plans and limits can change. The point is to understand which conveniences are cheap enough to use rather than optimize them away prematurely.

Service	Development / early use	Likely public-launch posture
Supabase	Free: 500 MB DB, 50k MAU, 1 GB	Pro from $25/mo for production

Service	Development / early use	Likely public-launch posture
	storage, 200 peak Realtime connections, 2M messages/mo.	features including 8 GB disk, 100 GB storage and daily backups retained 7 days.
Vercel	Hobby can support development/personal use.	Pro $20/mo, including $20 usage credit; Hobby is described as personal / non-commercial.
Upstash Redis	Free: 256 MB and 500k commands/mo.	Pay-as-you-go or fixed plan only when needed.
PostHog	First 1M product analytics events/mo and first 5k session recordings are free.	Usage-based after free tiers; install from day one.
Resend	Free: 3,000 emails/mo, 100/day, 1 domain.	Pro currently $20/mo for 50k emails when volume warrants.
Turnstile	Free plan with unlimited challenges for most production applications.	Likely remains free initially.

Where paying is worth it

- Database durability / backups once real advertiser money is involved.
- A serious payment processor and proper fraud controls.
- Legal review of the bounty, which matters far more than saving tens of dollars in infrastructure.
- Managed services that remove recurring operational chores from a small team.

Where not to add complexity

- No Kubernetes.
- No microservices.
- No separate event bus unless scale proves a need.
- No self-hosted analytics.
- No custom auth.
- No R2 while Supabase Storage is adequate.
- No proprietary Vercel database / KV dependency merely because Vercel offers it.

20. Alternatives considered

Next.js vs Vite + React + Hono

Dimension	Next.js	Vite + React + Hono
Mental model	More framework conventions / server-client concepts.	Very explicit frontend → API → DB model.
Public SEO / SSR	Convenient and integrated.	Possible, but requires deliberate SSR/prerender tooling.
Dynamic share images	First-class metadata / OG conventions.	Build separately.

Dimension	Next.js	Vite + React + Hono
App interactivity	Excellent.	Excellent.
Hosting portability	Good if avoiding proprietary services.	Excellent.
Integration work	Lower for this mixed public/app product.	Higher as features are assembled.

Community research was mixed rather than ideological. Recent Reddit discussions repeatedly praise Vite for simplicity and criticize unnecessary Next complexity, while others point out that once an app needs routing, SSR/SSG, API routes and metadata, developers often reassemble the conveniences a meta-framework already provides. For this product, the public discovery / social-sharing surface was the deciding factor.

Plain Postgres vs Supabase

Plain managed Postgres is fully viable and arguably architecturally purer. Supabase wins on the amount of plumbing removed - Auth, Realtime, Storage, dashboard and backups - while the critical SQL remains portable.

Self-hosted Redis on Contabo vs Upstash

The existing VPS could host Redis, especially because Redis is familiar. Upstash wins on zero-ops networking, TLS and serverless ergonomics. Since the Redis layer is non-authoritative, migration between providers is relatively low risk.

Gmail SMTP vs Resend

Gmail SMTP can work for development, but transactional delivery, logs, domain authentication and bounces are better delegated. Resend is a convenience purchase, not a technical necessity.

Vercel vs Contabo / other hosting

The VPS offers lower marginal hosting cost and a normal long-running Node process. Vercel wins on preview deployments and deploy/rollback ergonomics during rapid AI-assisted iteration. The current preference is Vercel first, while keeping the application portable.

21. Roadmap and decision backlog

Phase 0 - research / simulation

- Model attack-price dynamics and caps before implementing fixed numbers.
- Define what counts as a delivered advertising impression and qualified visit.
- Test candidate Audience Score formulas on synthetic / fixture data.
- Obtain initial legal opinion on promotional structures and target jurisdictions.
- Talk to payment providers with an accurate description of the model.

Phase 1 - visual prototype

- Build the desktop Throne scene with hardcoded products.
- Create 2-4 genuinely different visual directions in code.
- Select one and push it to a high art-direction standard.
- Design dethronement, bounty unlock and standings transitions.

- Lock golden screenshots across desktop / tablet / mobile.

Phase 2 - functional Season Zero

- Implement accounts, credits, transactional attacks and class caps.
- Add Realtime, Redis protections, delivery tracking and randomized evaluation.
- Implement season score and battle feed.
- Recruit real seed products.
- Enable bounty only if legal / processor blockers are cleared.

Phase 3 - repeatability

- Run multiple Indie seasons.
- Tune economy based on re-attack rate and advertiser ROI.
- Improve fraud controls and Audience Score.
- Only add Startup when Indie liquidity is healthy.

Phase 4 - Open and publisher inventory

- Add Open only when traffic can make uncapped spectacle rational.
- Prototype publisher embed.
- Test publisher RPM / revenue share.
- Validate that embedded inventory recruits challengers rather than only serving ads.

Decision backlog

Question	Why it matters	How to decide
Exact Indie cap?	Determines accessibility and revenue ceiling.	Simulate + first-season behaviour.
Decay function?	Controls vulnerability and pacing.	Game-economy simulation / observed reign length.
Guaranteed inventory method?	Protects advertiser value.	Compare shield vs impression guarantee.
Audience Score inputs?	Controls merit and fraud surface.	Start simple; validate against conversion quality.
Bounty legal structure?	Launch blocker for cash events.	Specialist counsel + processor approval.
Season length?	Controls urgency / fatigue.	A/B future seasons.
Publisher economics?	Controls network viability.	Pilot with a small set of publishers.

22. Competitor / community research notes

Milliboard

Milliboard publicly describes itself as King of the Hill: top positions are determined by the biggest single payment. At the time of research its page showed 0 contributors and $0 raised. This is not proof that the

broader concept fails; it is useful evidence that the bare paid-leaderboard primitive may be insufficient without real inventory, spectacle and audience value.

AdMogged

AdMogged is a close conceptual analogue: advertisers bid for one Mac menu-bar ad slot, and the highest live bid holds the slot as King of the Hill. It is particularly useful because the contested object is genuine inventory rather than a leaderboard row.

PromoteAnyth.ing

A January 2026 listing described PromoteAnyth.ing as King of the Hill meets Advertising: a $1-at-a-time public attention duel for the top promotional slot. Again, this supports the view that the auction primitive is discoverable by others and therefore unlikely to be a moat by itself.

Community view on Next.js vs Vite

Recent Reddit discussion is strongly split. Many developers argue that plain React + Vite is the better default when SSR/SEO is unnecessary and complain about Next.js framework complexity. Other comments point out that applications needing routing, SSR/SSG, API routes and metadata often end up adding the same pieces manually. One recent Vite + React + Hono thread praised faster development and fewer framework-specific debugging moments while acknowledging separate endpoints and weaker built-in SSR/SEO. This informed the choice rather than simply defaulting to Next.

Viktor Oddy / AI web-design workflow inspiration

Recent public posts from Viktor Oddy emphasize AI-assisted, high-end web creation with Claude Code / Opus 5 and GPT-5.6 Sol. The transferable lesson for this project is not to copy a particular aesthetic, but to use concrete references, deliberate asset / composition choices, coded visual iteration and motion as a first-class design problem. This document adapts that philosophy while intentionally omitting a mandatory Figma stage.

23. Source notes

External facts in this document were checked against the following sources on or near 8 August 2026. Pricing and product capabilities can change. Legal sources should be reviewed by counsel rather than treated as this document's interpretation.

1. Government of India - Promotion and Regulation of Online Gaming Act, 2025 - https://www.meity.gov.in/static/uploads/2025/10/8a7f103cefc68ed8aaa2ebc9a2ed7c13.pdf

2. MeitY - Promotion and Regulation of Online Gaming Act / Rules hub - https://www.meity.gov.in/documents/act-and-policies/promotion-andregulation-of-online-gaming-act-2025-and-its-corrigenda-kTMxQjMtQWa

3. PIB - Promotion and Regulation of Online Gaming Rules, 2026 - https://www.pib.gov.in/PressReleasePage.aspx?PRID=2254606&lang=1&reg=3

4. Stripe - Prohibited and Restricted Businesses FAQs - https://support.stripe.com/questions/prohibited-and-restricted-businesses-list-faqs

5. Milliboard - https://www.milliboard.com/

6. AdMogged - https://www.admogged.com/

7. PitchWall - PromoteAnyth.ing - https://pitchwall.co/product/promoteanything

8. Supabase pricing - https://supabase.com/pricing

9. Vercel pricing - https://vercel.com/pricing

10. PostHog pricing - https://posthog.com/pricing

11. Upstash Redis pricing - https://upstash.com/pricing/redis

12. Resend pricing - https://resend.com/pricing

13. Cloudflare Turnstile plans - https://developers.cloudflare.com/turnstile/plans/

14. Next.js - Metadata and OG images - https://nextjs.org/docs/app/getting-started/metadata-and-og-images

15. Next.js - Open Graph / Twitter image conventions - https://nextjs.org/docs/app/api-reference/file-conventions/metadata/opengraph-image

16. Vite - Server-Side Rendering guide - https://vite.dev/guide/ssr

17. Reddit r/nextjs - React+Vite vs Nextjs (March 2026) - https://www.reddit.com/r/nextjs/comments/1rkcqem/reactvite_vs_nextjs/

18. Reddit r/reactjs - Next.js / SPA Reality Check (July 2026) - https://www.reddit.com/r/reactjs/comments/1v0rmvb/nextjs_spa_reality_check/

19. Reddit r/webdev - Vite/React + Hono discussion - https://www.reddit.com/r/webdev/comments/1pkc3ke/i_ditched_nextjs_and_now_my_apps_navigation_are/

20. Viktor Oddy public profile / recent web-design tutorial posts - https://x.com/viktoroddy

Final note

The purpose of this document is to preserve the full shape of the opportunity without confusing enthusiasm with certainty. The most valuable next work is not adding more features to the concept. It is to simulate the economy, test whether the visual experience is magnetic, obtain a real promotion-law / processor answer, recruit a first set of legitimate products and measure whether dethronement causes advertisers to come back.

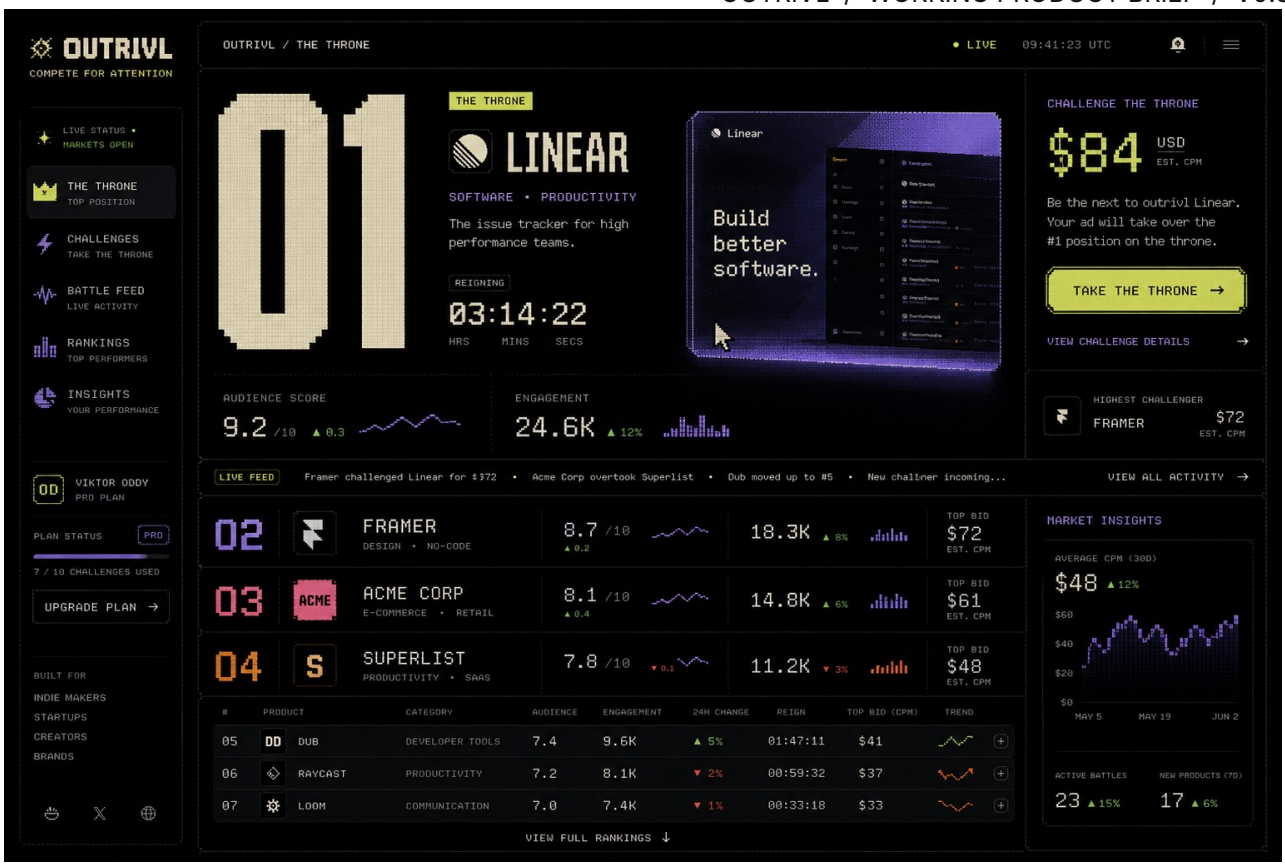
24. OUTRIVL brand and visual identity

Working name: OUTRIVL. The name carries competitive energy without forcing the product to look like a fantasy game. The practical product remains a discovery and advertising marketplace; Throne, Reign, Challenge, Crown Points and Battle Feed are system vocabulary layered onto it.

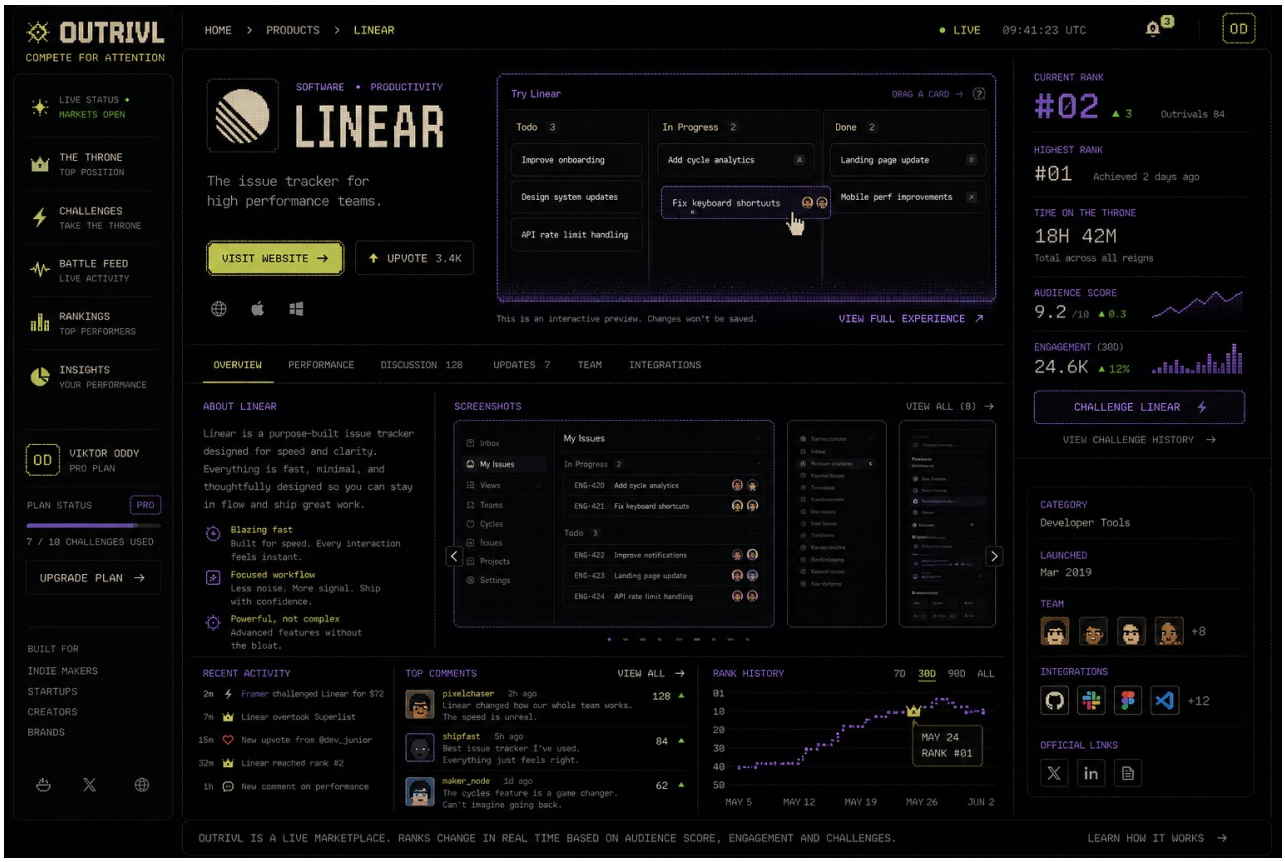
Visual thesis

The preferred direction is a new retro-computing / market-terminal aesthetic: the information density and live seriousness of Bloomberg, the product desirability and discovery logic of Product Hunt, and the art-directed composition / motion discipline associated with contemporary experimental web design. The reference is not an imitation of any one designer or brand. The aim is to create a recognizable OUTRIVL visual language.

- Dark near-black canvas with hard 1px structural rules rather than floating generic SaaS cards.
- A restrained retro-computing layer: bitmap-ish display type, terminal numerals, crisp cursor motifs, limited dithering / CRT texture, and occasional low-resolution edges - enough to feel nostalgic, not enough to become pixel art.
- A modern grotesk / clean sans-serif for ordinary reading so the nostalgic display layer remains special.
- Cream / warm-white as the principal text field, with small high-energy accents such as acid yellow-green, violet, orange or hot pink used semantically and sparingly.
- Dense data is desirable when hierarchy is strong. OUTRIVL should feel valuable because live state is visible, not because the page is empty.
- No literal throne chair, gold luxury SaaS aesthetic, gamer RGB glow, faux-casino ornament or generic dashboard tile wall.
- Rank is physical. #1 owns dramatically more canvas than #2; #2 more than #3; lower ranks collapse into compact market rows.
- Signature transitions occur only when state changes matter: dethronement, tier expansion / compression, bounty unlock and season resolution.



Golden reference A - current preferred Board direction: dense market information with a restrained retro-computing layer.



Golden reference B - product-page direction: permanent listing identity plus a large interactive product surface and public performance record.

29. Design implementation brief and golden references

Golden-screen principle

The two concept images above should be treated as direction references, not pixel-perfect implementation specifications. Browser screenshots become the source of truth. Codex should create coded visual directions, Playwright should capture fixed breakpoints, and visual iteration should be driven by explicit screenshot comparisons rather than vague adjectives.

- Board scene: #1 dominates the viewport; #2-#4 remain visibly competitive; the long tail becomes terminal-dense.
- Product scene: the product itself is the hero, with its interactive widget large enough to be genuinely useful rather than a decorative mockup.
- Do not let implementation drift back toward rounded generic SaaS panels. Hard grid, typography, rank scale and product artwork are the identity.
- Pixel / CRT effects are seasoning. Preserve legibility, accessibility, modern responsive behavior and excellent rendering on Retina displays.
- Motion should reinforce ownership transfer: the winner expands into space while the displaced incumbent physically compresses to its new rank tier.

25. Interactive listings and product pages

The newest product thesis is that OUTRIVL listings should not merely display products. Products should inhabit the page through controlled interactive experiences. A listing can contain a miniature useful / playful version of the product: a draggable issue board, before-after slider, calculator, sequencer, tiny design canvas, configurator, poll, interactive chart, mini-game, prompt demo or other approved interaction.

Why this changes the product

- It gives spectators a reason to browse advertising voluntarily; the ads themselves become things to do.
- It makes rank-based screen real estate economically meaningful. Higher rank buys more than pixels: it unlocks a richer product experience.
- It gives advertisers first-party signals beyond clicks: interaction starts, completion, dwell, feature use and downstream visits.
- It creates a distinctive browsing behavior that cannot be copied by merely adding bidding to a Product Hunt-style leaderboard.
- It remains useful even if the initial challenge / auction monetization underperforms; interactive product discovery can become a product category of its own.

Permanent listing page

Each company should have a persistent URL such as outrivl.com/p/<slug>. The page survives rank changes and becomes the canonical OUTRIVL identity for the product: full interactive experience, description, media, team / company information, categories, links, discussion and a public market-style performance record.

- Current rank and 24h movement.
- Highest-ever rank.
- Total Throne time and individual reign history.
- Audience Score and its trend.
- Widget interaction rate, completion and dwell metrics.
- Crown Points, CP/$ or other efficiency metrics where applicable.
- Challenge / takeover history and notable records.

- Rank-history timeline and shareable milestone moments.

26. Widget system and responsive canvas states

The widget belongs to the product listing, not to a single campaign. OUTRIVL renders different responsive manifestations of the same product experience according to rank and context.

State	Where used	Intent
ROW	Long-tail / compact ranking	Identity, one live metric, micro-action or non-interactive preview.
CARD	Mid-rank discovery	Compact interactive primitive with strict height / resource budget.
FEATURE	Podium / contender	Richer interaction, product media, meaningful in-place task.
THRONE	#1 premium canvas	Full approved mini-experience, strongest prominence and richest first-party engagement surface.
PRODUCT PAGE	Permanent listing	Full interaction independent of current rank, plus public history and product information.

Widget Kit - initial allowed primitives

- Text, labels, badges and product-controlled copy within moderation limits.
- Buttons / links with declared destinations.
- Tabs, accordions and carousels.
- Sliders, toggles, selects and controlled text inputs.
- Drag-and-drop within a bounded local state model.
- Before / after and reveal interactions.
- Simple calculators and configurators.
- Charts / metrics fed by advertiser-declared or OUTRIVL data.
- Canvas-like interactions using OUTRIVL-owned components.
- Media playback where licensed and performance-safe.
- Polls / choices / quizzes.
- Approved mini-game primitives when appropriate.

Security rule

Do not accept arbitrary advertiser JavaScript into the main OUTRIVL page. The default architecture should be declarative: advertiser intent → WidgetSpec → schema validation → policy validation → trusted renderer. Richer third-party embeds, if ever offered, should be separately sandboxed, permissioned and treated as a later security product.

27. AI-assisted visualization and widget authoring

AI should reduce the cost of creating great interactive listings without turning OUTRIVL into an uncontrolled code-hosting service. The model acts as a designer / compiler that emits a constrained specification.

Suggested authoring flow

11. Advertiser supplies product URL / description, screenshots or assets, brand tokens, desired call to action and an interaction goal.

12. OUTRIVL asks an OpenAI model for 2-4 interaction concepts appropriate to the product and available canvas states.
13. After selection, the model emits a strict WidgetSpec JSON object using only supported components, actions and data bindings.
14. Server validates the schema, checks destination URLs / content policy / resource budgets and rejects unsupported behavior.
15. Renderer composes trusted OUTRIVL components. No model-generated executable JavaScript is required for the default path.
16. Advertiser previews ROW / CARD / FEATURE / THRONE / FLOOR_BOOTH / PRODUCT_PAGE states and can refine the experience conversationally.
17. Approved versions are immutable / versioned for live use; edits produce a new version and can be A/B tested.

What a WidgetSpec could describe

- Layout regions and responsive state variants.
- Allowed visual components and hierarchy.
- Local interaction state and transitions.
- Declared analytics events such as widget_started, task_completed, cta_clicked.
- Data sources permitted for the widget: static advertiser content, OUTRIVL metrics, or explicitly approved backend endpoints.
- Fallback behavior for small screens, slow devices and reduced-motion preferences.
- Accessible names, keyboard behavior and text alternatives.

Visualizations

Charts and diagrams should follow the same pattern. The model should generate structured chart / diagram specifications from known data; OUTRIVL renders them. This keeps typography, accessibility, telemetry and performance consistent. Static promotional artwork can be generated / edited through the OpenAI image APIs and then reviewed before publication.

28. OpenAI Platform integration architecture

The OpenAI Platform is useful in OUTRIVL primarily as an authoring and creative-assistance layer, not as the authoritative runtime for money, ranking or scoring. Postgres remains economic truth. AI creates / edits widget specifications, explanatory copy, visual assets and optional interactive concepts.

Recommended service boundary

- Browser never receives the OpenAI API key.
- Next.js server route / backend worker owns OpenAI requests and applies per-account quotas and rate limits.
- Use an OUTRIVL-specific OpenAI project key so spend and access are isolated from unrelated projects.
- The authoring endpoint sends only the minimum product information needed for the requested generation.
- Use Structured Outputs / JSON Schema for WidgetSpec generation so the renderer receives machine-valid structured data rather than free-form code.
- Use image generation for advertiser creative assistance, listing art, share cards and optional widget textures / illustrations; keep generated media in OUTRIVL-controlled storage after moderation.
- Log prompt version, model, WidgetSpec version and approval state for reproducibility / debugging; do not mix these logs with the financial ledger.
- Apply spend caps and request budgets. AI generation is an authoring cost; it should not occur on every spectator page view.

Why structured output is the key mechanism

A model can be extremely creative while remaining constrained to an explicit JSON Schema. That allows OUTRIVL to expose a finite component vocabulary and still let advertisers describe experiences in natural language. The renderer - not the model - owns execution. This is the same architectural reason the interactive marketplace can be imaginative without becoming a security nightmare.

Image-generation path

For single-shot asset generation / editing, the Image API is the direct path. For a conversational listing studio where the advertiser iteratively edits a visual while the model keeps prior context, the Responses API image-generation tool is a better fit. Generated images should be stored, versioned, moderated and optimized before they become public assets.

Possible future AI functions - hypotheses, not V1 commitments

- Generate a widget concept from a product page and screenshots.
- Convert a product demo into several canvas states automatically.
- Suggest stronger micro-interactions based on observed abandonment without autonomously publishing changes.
- Generate alt text and accessibility metadata.
- Create share-card art for dethronements / milestones.
- Create chart / comparison views from advertiser or OUTRIVL metrics.
- Conversationally edit listing visuals while preserving the product brand system.

OpenAI documentation notes

Current OpenAI documentation recommends Structured Outputs rather than plain JSON mode when schema adherence is required. The image-generation guide distinguishes the Image API for direct image generation / editing from the Responses API for conversational or multi-step image workflows. These capabilities make a declarative AI-assisted widget studio technically plausible, but exact models, pricing and rate limits should remain implementation-time choices rather than frozen product assertions.

30. Addendum sources for v0.3

- OpenAI - Structured model outputs: https://developers.openai.com/api/docs/guides/structured-outputs
- OpenAI - Image generation: https://developers.openai.com/api/docs/guides/image-generation
- OpenAI - Model guidance / current model family: https://developers.openai.com/api/docs/guides/latest-model

31. AI commercial model: BYOK and OUTRIVL Cloud

OUTRIVL should separate the cost of AI authoring from the economics of the advertising marketplace. The preferred direction is BYOK as a first-class developer-friendly mode plus an optional managed OUTRIVL Cloud subscription for teams that want the infrastructure, billing and model access handled for them. Exact prices, included usage and provider support remain open parameters rather than fixed commitments.

Author-time AI versus runtime AI

This distinction is foundational. Most interactive listings should use AI only while the advertiser is creating or editing the experience. Once a WidgetSpec or reviewed SDK bundle is published, ordinary spectator interaction should execute locally in the OUTRIVL runtime without calling a model on every impression.

Runtime AI is a separate capability class for experiences whose product demonstration genuinely requires inference per visitor.

- Author-time AI: concept generation, copy refinement, WidgetSpec generation, image generation / editing, responsive-state derivation and creative experimentation. Cost occurs during creation, not every page view.
- Runtime AI: product assistants, transformations, prompt demos or other model-backed experiences invoked by spectators. These require explicit quotas, latency handling, abuse controls and spend budgets.
- Neither author-time nor runtime AI decides credits, Throne ownership, Crown Points or other economic truth. Those remain deterministic server-side systems backed by Postgres.

BYOK mode

- Advertiser connects a dedicated provider / project API key and pays the model provider directly for inference.
- BYOK should be presented as a legitimate power-user choice, not a hidden cheap tier: bring your own model credentials, preserve direct spend visibility and revoke independently.
- Secrets are encrypted at rest and used only by OUTRIVL server-side services. They never enter the public widget bundle, DOM, browser storage or client network trace.
- For OpenAI, encourage a dedicated OUTRIVL project key rather than reuse of a personal all-purpose key, so usage and revocation remain isolated.
- Provider abstraction may later support multiple model providers, but the first implementation can remain narrower if that produces a safer and better product.

OUTRIVL Cloud

The paid managed tier should sell convenience and creative infrastructure, not merely resell tokens. It can bundle managed model access, generation allowance, image tools, version history, team workflows, model routing, analytics and higher creative limits. Subscription pricing can coexist with variable usage for genuinely inference-heavy runtime widgets.

- No external API-key setup.
- Managed AI widget / visualization authoring.
- Image generation and editing assistance.
- Automatic ROW / CARD / FEATURE / THRONE / FLOOR_BOOTH / PRODUCT_PAGE derivations.
- Version history, previews and restoration.
- Creative variants / A-B experimentation.
- Brand-kit context and reusable product assets.
- Usage dashboard, hard budgets and predictable account limits.
- Team collaboration and approval workflows as a higher-tier capability.

Revenue resilience

This creates a revenue stream that is independent of whether competitive challenge spend alone becomes sufficiently large. OUTRIVL can potentially monetize competitive inventory, managed creative infrastructure, and later distributed / publisher-side interactive placements. These are hypotheses to validate separately; the brief should not assume all three succeed.

32. Public Widget SDK and GitHub developer platform

OUTRIVL should have a public developer platform for companies that want experiences beyond the declarative WidgetSpec system. The public repository becomes the contract between OUTRIVL and widget

developers: SDK, simulator, CLI, examples, permission model, performance rules, accessibility guidance and submission workflow.

Proposed repository shape - illustrative

github.com/outrivl/widget-sdk

- @outrivl/widget-sdk - core runtime bridge and types.
- @outrivl/widget-react - optional convenience bindings if React is supported inside the SDK sandbox.
- create-outrivl-widget / CLI scaffolding and validation tools.
- /examples - calculator, kanban demo, before-after, configurator, mini-game, product tour and runtime-AI examples.
- /docs - permissions, responsive states, events, network policy, accessibility, submission and review.

Developer loop

A developer should be able to scaffold a widget locally, run an OUTRIVL simulator and switch instantly among ROW, CARD, FEATURE, THRONE, FLOOR_BOOTH and PRODUCT_PAGE. The simulator should expose the same size constraints, capability bridge, telemetry hooks and resource budgets as production so a widget that passes locally is unlikely to fail after submission.

- Scaffold → develop locally → simulate all canvas states → validate → submit → automated checks → preview → review → signed release.
- Golden example widgets should be open source and deliberately high quality so the ecosystem has a strong design bar from the beginning.
- Agentic coding tools should be able to read the public SDK documentation and generate compliant widgets without needing proprietary internal knowledge.
- A later Widget Gallery can expose reusable templates / starter kits that advertisers can fork and customize.

Three trust levels

- Native WidgetSpec - default and safest path. Declarative, rendered entirely by OUTRIVL-owned primitives, suitable for broad AI-assisted generation.
- SDK Widget - custom code executed only inside an isolated sandbox with constrained capabilities, strict budgets and review.
- Privileged Integration - exceptional capability class for unusual products; manually reviewed, explicitly permissioned and expected to be rare.

The purpose of these tiers is not to constrain creativity arbitrarily. The constraint itself can become part of OUTRIVL's product identity: developers know the canvas, APIs and limits and build inventive experiences within them.

33. Widget security, permissions, review and distribution

Custom widgets must never inherit the privileges of the OUTRIVL host application. A listing is untrusted advertiser content even when the advertiser is reputable. Security therefore belongs in the product architecture, not merely in moderation policy.

Sandbox boundary

- Custom SDK widgets should execute in a separate-origin sandboxed iframe or equivalent isolated runtime with a restrictive Content Security Policy.

- No direct access to OUTRIVL cookies, authentication state, host DOM, privileged storage, internal APIs or payment / ranking state.
- No arbitrary top-level navigation, popups, background tasks or hidden persistent processes.
- Communication with OUTRIVL occurs through a narrow versioned capability bridge rather than ambient browser authority.

Capability manifest

Every SDK widget requests explicit capabilities in a manifest. A manifest is a request subject to policy and review; it is not self-granted permission. Example capability families include interaction, analytics events, safe navigation, public listing context, approved AI invocation, declared network access and resize / canvas requests.

- outrivl.analytics.track(...) - emit declared first-party events.
- outrivl.navigation.openProduct(...) / openExternal(...) - navigation mediated by OUTRIVL.
- outrivl.ai.invoke(...) - server-side model call using advertiser BYOK or managed Cloud entitlement; the key never reaches the widget.
- outrivl.data.getPublicContext(...) - obtain only explicitly public listing / rank / season data.
- outrivl.network.request(...) - optional mediated request through an allowlisted server proxy.
- outrivl.ui.resize(...) - request bounded layout changes consistent with the current canvas state.

Network and secret handling

Do not initially allow unrestricted fetch() to arbitrary internet destinations. Where external data is genuinely required, the widget declares approved origins and OUTRIVL mediates or proxies the request. This enables domain allowlisting, rate limits, secret isolation, logging, abuse detection and emergency revocation. Provider keys, OAuth tokens and private API credentials remain server-side.

Hard resource budgets

- Bundle / asset size.
- Startup time and time-to-interactive.
- CPU / long-task budget.
- Memory budget.
- Animation / frame-rate budget and reduced-motion behavior.
- Network request count and response-size budget.
- Runtime model-call count / spend budget.
- Media weight and autoplay policy.

A visually impressive widget that materially degrades the visitor's device or the Board is not acceptable. Resource limits should be observable in the local simulator and enforced in production.

Fallbacks and accessibility

- Every widget must provide a static or low-capability fallback so the listing remains useful when execution fails or is disabled.
- Keyboard interaction, visible focus, accessible labels and text alternatives are required for supported interactive primitives.
- Respect reduced-motion and device capability where possible; nostalgia / visual experimentation is not an excuse for inaccessible interaction.

Review and release pipeline

- CLI validation before submission.
- Automated dependency, malware, static-analysis, CSP, permission, accessibility and performance checks where technically feasible.
- OUTRIVL-hosted preview environment using the real runtime constraints.

- Human review for new custom-code widgets and any elevated capability class.
- Immutable, versioned releases such as product-widget@1.4.2; live deployments reference an approved version.
- Advertisers cannot obtain approval for harmless code and then silently replace the reviewed JavaScript from their own server.
- OUTRIVL retains an emergency kill switch and can roll back to a previous reviewed version or static fallback.

Why GitHub matters strategically

The GitHub repository can become both documentation and distribution infrastructure. Open-source reference widgets lower the cost of entry, give coding agents concrete examples, encourage forks and community templates, and make OUTRIVL feel like a developer platform rather than a closed ad builder. The long-run hypothesis is an ecosystem of small interactive product experiences that are safe enough to run in a shared public marketplace.

34. v0.4 change note

v0.4 adds the commercial and developer-platform implications of interactive listings: BYOK plus optional managed OUTRIVL Cloud, the author-time / runtime AI distinction, a public Widget SDK / GitHub strategy, explicit widget trust tiers, sandboxed capabilities, mediated networking, secret isolation, resource budgets, versioned reviewed releases, static fallbacks and emergency kill switches. These are architecture directions and hypotheses, not frozen implementation claims.

OUTRIVL WORKING PRODUCT BRIEF - v0.4 - BYOK / Cloud and Widget SDK platform addendum

35. Agent execution and handoff playbook

Purpose. This section is written so Codex, Claude Code, Claude Design, or a future coding/design agent can open the repository with this brief and continue without the founder having to restate the product. It defines the order of work, role boundaries, visual constraints, handoff artifacts, and the definition of done for the first build cycle.

Source-of-truth order

- This brief is the product and architecture source of truth. If an agent preference conflicts with a documented decision, the brief wins unless the founder explicitly changes it.
- Approved golden reference images are the visual source of truth. They define mood, hierarchy, density, pixel treatment, and the amount of nostalgia better than generic design-system conventions.
- The live coded prototype becomes authoritative only after it has been explicitly approved as a new golden state. An agent must not silently redesign approved surfaces while refactoring.
- Agent inference is last. When something is genuinely unresolved, make the smallest reversible choice and record it rather than inventing a new product direction.

Tool roles

Tool	Primary responsibility	Do not let it drift into
Claude Design	Art direction, reference interpretation, composition studies, visual-system exploration, typography/grid/motion concepts. It may deliberately ignore implementation difficulty while exploring.	Production architecture, backend choices, or quietly normalizing the design into familiar SaaS patterns.
Codex	Visual implementation in the browser. Build the approved compositions, create variants when asked, use hardcoded data early, run the app, capture screenshots, and iterate toward the golden references.	Premature abstraction, backend work before the visual grammar is proven, or substituting generic components for difficult visual details.
Claude Code	Production engineering after a visual direction is approved: component boundaries, Supabase/Postgres logic, widget runtime, SDK, permissions, security, tests, refactors, deployment, and maintainability.	Redesigning the approved interface during refactors or replacing bespoke visual behavior with a generic library for convenience.
OpenAI image generation / API	Asset manufacture, concept exploration, listing creative, textures, campaign imagery, iterative image edits, and later advertiser-facing AI authoring.	Being the source of economic truth, ranking truth, or arbitrary executable widget code.

First build sequence - do this in order

0. Repository shell: Create the Next.js + React + TypeScript + Tailwind repository. Do not add Supabase, Redis, payments, auth, or other infrastructure yet unless required to make the visual prototype run.

1. Reference pack: Create /references and place the approved OUTRIVL Board and Product Page concepts there using stable names such as outrivl-board-golden.png and outrivl-product-page-golden.png. Place this brief in /docs. Add a short /docs/design-constitution.md distilled from Sections 24-29 and this section.

2. Board / Throne: Build the desktop Board first with hardcoded products and fake live data. #1 must physically dominate the page; #2 and #3 must be materially smaller; lower ranks progressively collapse into dense rows. Include one genuinely interactive miniature product widget.

3. Product Page: Build the permanent product page in the same visual language. It must feel like the same system without becoming a conventional SaaS detail page. The full widget, market/performance history, product information, and clear external CTA all belong here.

4. Dethronement scene: Prototype the signature state change: incumbent compresses, challenger expands into the #1 canvas, rank geometry reorders, timer and attack price update, and the live tape records the event. Motion should be theatrical at the moment of state change and calm otherwise.

5. Widget interaction scene: Prove that OUTRIVL listings are software, not static ads. Build at least one miniature interaction that is enjoyable without leaving the page and demonstrates the responsive canvas states.

6. Spatial / Floor exploration: After the Board, Product Page, dethronement and one interactive widget are visually convincing, prototype one optional spatial scene using the same product/rank data. The goal is to test whether an inhabitable attention market adds spectator value without making information retrieval slower. Do not let this exploration delay the core Board.

7. Advertiser Widget Studio: Prototype the authoring experience: product context and assets in, widget concept/spec out, previews for ROW / CARD / FEATURE / THRONE / FLOOR_BOOTH / PRODUCT_PAGE, and clear distinction between native WidgetSpec and SDK widgets.

8. Visual lock: Only after the above scenes are approved, promote screenshots to golden references and freeze the first visual grammar. Then begin productionization.

9. Claude Code productionization: Introduce real domain models, Supabase/Postgres, Realtime, Redis where justified, auth, credits, attack transactions, telemetry, widget sandboxing, SDK packaging, tests, observability, and deployment. Preserve the approved visual output through regression screenshots.

Six golden scenes

- Board / Throne - the recognisable public market surface and primary visual identity.
- Product Page - the durable identity page and full interactive showcase for one company.
- Dethronement - the signature transition proving rank physically changes the page.
- Widget Interaction - a product miniature that does something useful or delightful in place.
- Spatial Floor - an optional live world in which products occupy ranked spaces, attention is physically legible, and entering a booth activates the same OUTRIVL widget system. This is a mechanic exploration, not permission to become an MMO.
- Advertiser Widget Studio - the creation and preview surface for native/AI-assisted widgets.

Visual non-negotiables for agents

- Nostalgic computing, not literal 8-bit pixel art. The approved concept has the correct amount of pixelation: enough to create a recognisable texture and typography language, not enough to make the product look like a game UI skin.
- The design should feel new rather than retro cosplay. Use the same principle that makes Nothing products distinctive: a coherent proprietary visual language built from familiar technological memory, not a direct imitation of Nothing or any other brand.
- Bloomberg is the information-density reference: tiny legible metrics, live tape, sparklines/deltas/timestamps, market-state language, and dense lower ranks. It is not permission to copy Bloomberg visually.
- Avoid generic AI-SaaS tells: excessive rounded cards, glassmorphism, soft purple gradient glows, generic left sidebars, pill overload, oversized empty KPI cards, and stock dashboard layouts.
- Use hard 1px geometry, unusual editorial proportions, deliberate negative space, restrained texture/dither/scanline motifs, and a distinctive display type treatment paired with a highly legible text face.

- Rank is layout. #1 receives the most physical canvas; #2/#3 receive visibly less; lower ranks compress. Do not reduce rank to a badge next to identical cards.
- Products are the protagonists. OUTRIVL chrome should frame product experiences rather than overwhelm them.
- The Throne is product terminology, not medieval art direction. No literal throne chair, crowns everywhere, swords, faux heraldry, or fantasy-sports styling.
- Motion is scarce and meaningful. Use it for takeovers, rank-tier expansion/compression, bounty unlocks, and other state changes. Routine browsing should remain fast and calm.

Browser-as-design-canvas loop

For the visual sprint, the running browser is the design canvas. Figma is not required. The loop is: **reference → coded composition → fixed screenshot → compare → annotate → revise → promote to golden.**

- Use hardcoded fake data until the scene is visually approved. Fake data is preferable to backend complexity during composition work.
- Capture fixed screenshots at agreed breakpoints after each meaningful visual iteration. Desktop first; add mobile once the desktop grammar is stable.
- Compare screenshots to the approved reference and fix hierarchy, spacing, typography, texture, density, and motion before extracting abstractions.
- Once a scene is golden, keep its screenshots as visual-regression fixtures so engineering refactors cannot quietly sand away the design.

Copy-paste starter brief for Codex

Read the full OUTRIVL brief, especially Sections 24-35, and inspect every file in /references before coding. This is a design-implementation task first, not a production-architecture task. Build the desktop Board with hardcoded fake data. Match the approved reference direction: near-black canvas; restrained retro-computing character; light pixel influence rather than pixel-art UI; distinctive display typography; hard 1px geometry; subtle texture/dithering; Bloomberg-like information density; minimal generic rounded SaaS cards. Rank must physically determine screen real estate. #1 must dominate and contain a functioning miniature product interaction; #2 and #3 must be materially smaller; lower ranks must progressively collapse into dense market rows. Do not add backend, auth, database integration, payment logic, generic component libraries, or abstractions merely because they may be useful later. First make one exceptional page. Run it, capture screenshots, compare against the golden references, and iterate until the page is recognisably OUTRIVL even with the logo hidden.

Copy-paste productionization brief for Claude Code

Treat the approved OUTRIVL prototype and golden screenshots as visual contracts. Read the full brief before changing architecture. Your job is to productionize rather than redesign: preserve the rendered output while introducing maintainable component boundaries, domain types, Supabase/Postgres, transactional attack/credit logic, Realtime, Redis only for non-authoritative fast state, PostHog/Sentry, widget sandboxing, the public Widget SDK, BYOK/OUTRIVL Cloud server boundaries, tests, and deployment. PostgreSQL remains authoritative for economic truth. Widgets never receive provider keys or host-page privileges. If an engineering refactor changes a golden scene, treat that as a regression unless the founder explicitly approves a new design.

Required handoff artifacts in the repository

- /docs/OUTRIVL-brief.pdf or .docx - this document or its current successor.
- /docs/design-constitution.md - concise design rules and anti-patterns distilled from the brief.
- /references/outrivl-board-golden.png - approved Board visual target.
- /references/outrivl-product-page-golden.png - approved Product Page visual target.
- /references/founder-floor-mechanic-reference.png - mechanic inspiration for an inhabitable directory / live founder floor. Reference only; OUTRIVL must not copy its visual treatment, booth design, avatar style, wording, or brand expression.

- /references/golden/ - approved coded screenshots by breakpoint and scene.
- /docs/widget-runtime.md - WidgetSpec, SDK trust tiers, capability bridge, budgets, network/secret rules, review pipeline.
- /docs/decisions/ - short ADR-style notes for decisions that materially change the brief.
- README.md - exact commands to install, run, test, capture screenshots, and validate widgets.

Definition of done for the first visual milestone

- A fresh agent can clone the repo, read the brief/reference pack, run one documented command, and see the Board locally.
- The Board is recognisably OUTRIVL with the logo hidden; it does not look like a generic SaaS dashboard or gaming skin.
- Rank visibly controls physical canvas size.
- At least one listing contains a real local interaction, not a static mock image.
- The Product Page uses the same visual grammar and contains the full interactive experience.
- The dethronement transition has been demonstrated with fake/local state.
- Golden screenshots exist and are stable enough to use as regression references.
- No production backend work has forced premature compromises into the visual design.

Explicit anti-goals during the first sprint

- Do not start by building auth, billing, database migrations, admin panels, publisher infrastructure, or a complete design-system library.
- Do not ask a coding agent to invent the brand from scratch after approved references exist.
- Do not replace difficult bespoke composition with a component kit simply to move faster.
- Do not let Claude Design, Codex, or Claude Code independently reinterpret OUTRIVL into three different products. They share one brief and one golden reference set.
- Do not treat the first coded prototype as throwaway. It is a discovery instrument that should evolve into the production visual system after approval.

36. v0.5 change note

v0.5 adds an explicit agent-operating layer so the project can be handed directly to Claude Design, Codex and Claude Code. It defines tool responsibilities, source-of-truth precedence, the first repository/build sequence, five golden scenes, visual anti-drift rules, browser-based screenshot iteration, copy-paste starter briefs for Codex and Claude Code, required repository handoff artifacts, and the definition of done for the first visual milestone. No core economic, widget-security, BYOK/Cloud, or architecture decisions from v0.4 are removed.

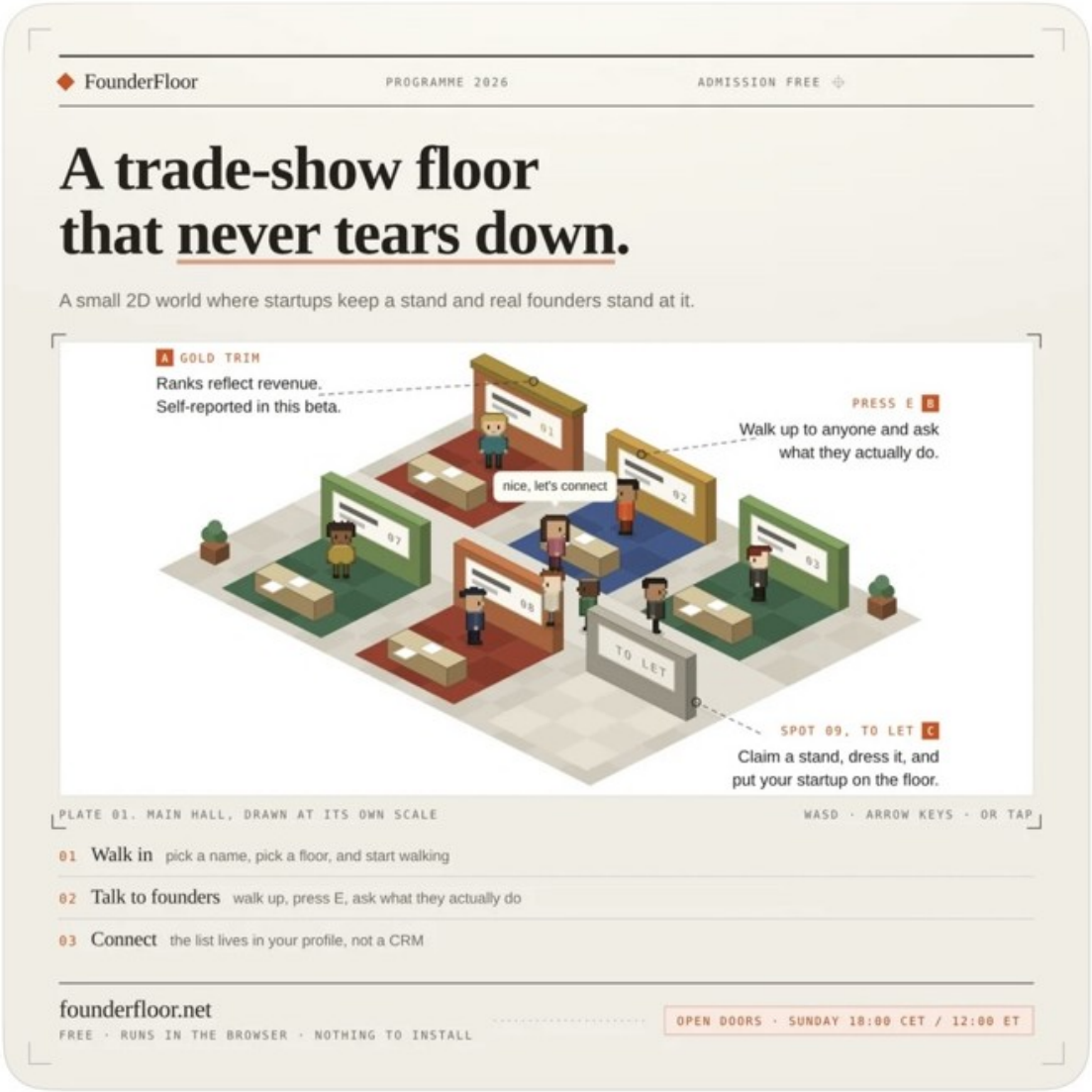
OUTRIVL WORKING PRODUCT BRIEF - v0.5 - agent execution and handoff playbook

37. Spatial discovery mode - The Floor

Status: approved product direction for exploration; optional surface, not a replacement for the Board. **Core principle:** make the directory inhabitable while keeping OUTRIVL fast, legible and useful as an information product.

Mechanic reference - Founder Floor

Reference captured from a founder-shared Founder Floor concept. The useful idea is not its exact graphics. The useful idea is that a directory / trade-show list becomes a place: companies occupy stands, people can move through the environment, founders can be present, and discovery happens through spatial proximity and interaction rather than through static cards alone.



REFERENCE 37-A - Founder Floor mechanic reference supplied by the founder. Use as interaction inspiration only, not as an art-direction target or copy source.

OUTRIVL translation. The Board remains the serious, high-density attention market. The Floor becomes a second lens over the exact same products, ranks, classes, widgets and market events. A visitor who wants speed can stay on the Board; a visitor who wants discovery, spectacle or live founder interaction can enter the Floor. The product must never force spatial navigation to reach essential information.

Design thesis - an inhabitable attention exchange

- Do not build a generic metaverse, virtual office, pixel MMO, or novelty game. The Floor should feel like an alternate-history software exchange: as if a 1990s computer culture imagined a live market for software attention and it matured into a modern product.
- Preserve OUTRIVL's established aesthetic: near-black ground, cream/off-white display type, sparse signal yellow/green/violet/orange, restrained pixel texture, hard 1px geometry, terminal-density data, subtle dithering/scanline memory, and modern editorial composition. The spatial layer adds depth and movement without abandoning the brand.
- The Founder Floor reference is about inhabitable mechanics, not appearance. Do not copy its beige trade-show board, booth geometry, sprite style, typography, text, layout or branding.
- The spatial scene should remain readable as a market. Every animated or decorative element should map to product state, rank, attention, presence, a challenge, a launch, or another real event whenever possible.

One data model, three public surfaces

Surface	Primary job	Interaction density	Rule
BOARD	Fast market scanning, ranking, challenges, market state and discovery.	Dense information; one or a few active widgets.	Default serious interface. Never degraded to make the Floor necessary.
FLOOR	Spatial discovery, live presence, spectacle, launches and serendipity.	Many lightweight booth states; richer experience loads on focus/entry.	Optional second lens over the same authoritative state.
PRODUCT PAGE	Permanent identity, full interactive showcase, history, information and conversion.	Highest per-product depth.	Canonical product URL regardless of current rank.

Canonical identity. A product has one product ID and one permanent URL. Board cards, Floor booths and the Product Page are renderings of that same entity. Rank, Audience Score, challenge state, founder presence and approved widget versions should not fork into separate systems.

Rank should become physical space

- #1 / Throne: a dominant central installation or landmark, visually impossible to miss. It receives the largest spatial footprint and the richest FLOOR_BOOTH treatment. The Throne is expressed through scale and attention, not medieval props.
- #2-#3: substantial adjacent installations with clearly reduced footprint. They should feel like legitimate contenders, not identical stalls with different number labels.
- #4-#10: medium stands / terminals. Each is distinct enough to inspect and interact with, but the hierarchy is visibly compressed.
- Lower ranks: progressively smaller presences. The scene should not try to render hundreds of equally detailed booths. Use compact terminals, grouped rows, teleport/search results, district boards or off-screen discovery mechanisms.
- Rank changes can rearrange the environment, but transitions must be intelligible. Large jumps may be staged or interpolated rather than teleporting every object violently on every update.
- The three competitive classes remain separate economies. The visitor switches INDIE / STARTUP / OPEN; the Floor must not merge their ranks into one fake hierarchy. Categories such as AI, DevTools and Productivity may influence districts / visual clustering, but remain filters and metadata rather than separate arenas.

Booths are interactive product surfaces, not posters

New renderer state: FLOOR_BOOTH. The same listing experience should now support ROW → CARD → FEATURE → THRONE plus FLOOR_BOOTH and PRODUCT_PAGE. The Floor version is deliberately lightweight until a visitor focuses it.

- Idle booth: logo / name / rank / one signal metric / small animated affordance. Keep runtime cost tiny.
- Approach / focus: reveal a condensed explanation, current challenge state, founder presence and one obvious interaction cue.
- Activate: load the company's approved interactive widget inside the booth, a foreground panel, an in-world console, or a portal-like overlay. Example: drag a Linear ticket; adjust a finance calculator; play a synth; scrub a before/after; try one model transformation.
- Deep inspect: transition to the permanent Product Page without losing context. The visitor should never need to navigate a maze to find screenshots, pricing context, links or product information.
- Performance rule: do not fully boot every company's rich widget at once. Most booths remain cheap representations; richer code and assets lazy-load only for nearby/focused/activated products.
- Fallback rule: every booth has an attractive static representation if custom code fails, the device is weak, reduced-motion is enabled, or the widget is disabled by policy.

Founder presence and live office hours

The compelling social primitive is presence attached to a product, not a generic social feed. A product can feel inhabited when a verified founder / team member is actually available.

- FOUNDER ONLINE / TEAM ONLINE state on the booth and Product Page, with an optional short status such as "Ask me about our API" or "Hiring founding designer".
- Scheduled office hours: founders can announce a 20-60 minute window when they will be at the booth. OUTRIVL can surface these on the live tape and discovery modules.
- Lightweight ask / connect interaction. Start with asynchronous questions or bounded live Q&A rather than building a full Discord replacement.
- Live launches: a company can schedule a launch window, turn on its richer interactive demo, answer questions and create a reason for people to arrive simultaneously.
- Optional opportunity badges such as LOOKING FOR COFOUNDER, HIRING, OPEN TO PARTNERSHIPS or SEEKING DESIGN PARTNER. This creates a path toward founder matching without turning OUTRIVL into a swipe app.
- Presence must be opt-in, rate-limited and abuse-aware. Do not expose personal location, exact idle state or private contact data. A team can appear "present" without revealing which employee is physically active.

Make attention physically visible

This should become a signature OUTRIVL behavior. The environment can visualize aggregated real attention so the spectator literally sees where the market is flowing.

- Visitor energy: subtle pulses, particles, scanlines, light trails, crowd density, terminal activity or other abstract signals can increase around products receiving real interactions.
- Do not render fake bustle. Environmental activity should derive from actual aggregated events such as booth focus, widget starts, meaningful interaction, dwell bands or CTA continuation.
- Protect privacy and avoid exposing tiny cohorts. Smooth / bucket / delay public activity, hide exact counts below thresholds, and never let an observer infer a named person's behavior from a visual particle.
- Distinguish raw attention from quality. A booth may look busy while its Audience Score remains mediocre; another may have fewer visitors but strong interaction depth. The data labels should make that distinction legible.
- The same events feed analytics: floor impression → booth focus → widget start → meaningful interaction → product-page open → external CTA. These become useful first-party metrics for advertisers.

Physicalize market events

- Challenge incoming: a restrained indicator can appear near the target installation; do not turn routine bidding into combat animation spam.
- Dethronement: the incumbent installation compresses / dims, rank markers update, the challenger expands into the #1 footprint, attention signage retargets, and the live market tape records the takeover. This should reuse the same signature motion language as the Board.
- Rank movement: booths shift size / position or visual prominence according to meaningful tier changes. Minor single-rank noise should not constantly rearrange the entire floor.
- Bounty unlock: if promotional bounty mode is active, a floor-wide board / signal may update. The event is celebratory but does not imply that paid attacks improve the legal promotional score.
- Market open / season end / champion: the environment can support scheduled ceremonies and recap states without making the normal product feel like fantasy sports.

Movement without friction

- Desktop may support click-to-move, arrow/WASD movement, pan/drag, or direct teleport; the prototype should test which feels least gimmicky. Movement is an exploration affordance, never a gate.
- Always provide search, category filters, rank jump, "go to #1", active-founder list, live-launch list and recently active products. A visitor should reach a known product in seconds.
- Inspect mode should work from a click/tap without requiring the avatar to stand on an exact pixel.
- Mobile should prioritize tap-to-inspect / pan / jump controls rather than pretending a phone is a desktop game controller.
- Reduced-motion users receive instant state changes or gentle fades. Keyboard and assistive-technology users must be able to access the equivalent product list and actions without operating the spatial scene.
- The Board is the accessibility and information-equivalence fallback. No essential content or action may exist only inside the Floor.

Visitor presence - social, but not creepy

- Default public presence should be abstract: anonymous cursors, sparks, silhouettes or aggregate crowd marks. Do not require a visible avatar.
- Authenticated users may opt into a lightweight public identity / handle. Visibility must be explicit and reversible.
- Avoid follower counts, popularity vanity loops and generic feed mechanics at launch. OUTRIVL social interaction should remain attached to products, launches, questions and useful founder presence.
- Direct messaging is not required for the first version. A bounded "request intro / connect" action can route through controlled contact preferences later.
- Moderation controls are mandatory before any open chat: block/report, rate limits, product-owner moderation for booth sessions, global enforcement and event-level kill switches.

Technical implementation hypothesis

Do not create a second backend. The Floor is a client rendering of the same authoritative OUTRIVL state. PostgreSQL remains economic truth; rank/challenge transitions are decided server-side; Realtime/Broadcast distributes public events; the spatial client animates them.

- Route / surface: a lazy-loaded client-heavy /floor experience inside the existing Next.js product. It must not increase the loading cost of ordinary Board/Product Page visits.
- Scene renderer: prototype with the lightest technology that achieves the art direction. DOM/CSS may be enough for a diagrammatic 2D layout; PixiJS is a reasonable working hypothesis if many sprites, particles and smooth 2D/2.5D movement are needed. Do not add a heavy 3D engine by default.
- React remains responsible for accessible overlays, inspection panels, filters, search, product metadata and route transitions. The scene renderer should not become the source of truth for application state.
- Presence is ephemeral and non-authoritative. Redis / Realtime presence may hold transient session state; never write every cursor movement into PostgreSQL.

- Server broadcasts semantically meaningful events such as PRODUCT_RANK_CHANGED, THRONE_TAKEN, FOUNDER_PRESENCE_CHANGED, LIVE_LAUNCH_STARTED, BOUNTY_UNLOCKED. Clients decide how to animate them.
- Widget sandbox rules from Sections 30-33 remain unchanged. FLOOR_BOOTH does not grant custom code extra browser privileges. Rich widgets load through the same signed, reviewed runtime.
- Resource budgets need a Floor multiplier: cap simultaneously active widget instances, particles, network calls and asset loads. Favor pooled/recycled scene objects and coarse public presence updates.

Example visitor session

1. Enter: Visitor opens OUTRIVL / Floor in the INDIE class. The market is already alive: #1 is a large central product installation, #2/#3 flank it, and nearby stands show current attention signals.

2. Notice: A live tape says a founder is online at a DevTools product and another company is launching in 12 minutes. The visitor can walk, click or jump directly.

3. Approach: The booth reveals its condensed product identity, rank, Audience Score and an interaction cue.

4. Interact: The visitor activates the miniature product widget. A real product behavior occurs locally in the sandbox.

5. Inspect: The visitor opens the permanent Product Page for full information, performance history, discussion and external CTA.

6. Return: A challenge fires elsewhere. The central installation changes hands and the Floor visibly reorganizes, giving the visitor a reason to look back at the market rather than immediately leave.

What would prove the Floor is useful

- Higher product discovery depth: visitors inspect more unique products per session than on the Board alone without materially lowering conversion quality.
- Meaningful widget activation rate from the spatial surface, not merely avatar movement.
- Return behavior around live launches, founder office hours and major rank changes.
- Founder participation: teams voluntarily schedule presence because it produces useful conversations / traffic.
- Spatial-to-product-page continuation and external CTA rates remain healthy; the world is not trapping people in novelty interaction.
- Performance stays excellent on ordinary laptops and modern phones. If the world materially harms load time, battery or accessibility, simplify it.
- Qualitative test: users describe the Floor as a new way to discover products, not as "a cute pixel game bolted onto Product Hunt."

Scope boundaries - do not let this eat the company

- The Board, Product Page, interaction runtime and advertiser value proposition remain higher priority than a complete spatial world.
- Prototype one coherent Floor room / market before inventing multiple cities, lands, floors, avatars, inventories or collectible systems.
- No virtual real estate speculation, purchasable cosmetic land, NFT logic or artificial scarcity unrelated to actual advertising inventory.
- No requirement for users to create avatars before browsing.
- No full social network, voice chat or unmoderated global chat in the first spatial release.
- Do not couple the ranking engine to scene coordinates. Rank determines presentation; the economic model should remain testable without the Floor.
- If the Floor fails, OUTRIVL still works. That is a feature of the architecture, not a lack of conviction.

Agent brief - spatial prototype

Read the OUTRIVL brief and approved Board/Product Page references first. Then inspect the Founder Floor reference only for the mechanic of an inhabitable directory. Do not copy its aesthetic. Prototype a single optional OUTRIVL Floor for one competitive class using hardcoded products and fake presence. Preserve the established nostalgic-terminal visual language. #1 must dominate physical space; #2/#3 must be materially smaller; lower ranks compress. Booths are lightweight until focused; activating one loads the same interactive widget system used elsewhere. Show founder-online state on at least one product and visualize attention using aggregated activity rather than fake decorative crowds. Add direct search/jump/inspect so no user is forced to walk. Demonstrate one dethronement and one live-launch/founder-presence event. Keep the scene client-side and disposable; do not build new backend infrastructure until the interaction is visually and behaviorally approved.

Reference-pack rule

Store the supplied Founder Floor crop as /references/founder-floor-mechanic-reference.png and annotate it in README or design-constitution as **MECHANIC REFERENCE - NOT VISUAL TARGET**. Approved OUTRIVL Board and Product Page images remain the visual targets. If a coding agent makes the Floor beige, cute-isometric, or visually derivative of the reference, reject the direction.

38. v0.6 change note

v0.6 adds the optional Spatial Discovery Mode / Floor as a detailed product and implementation direction. It records the Founder Floor post as a mechanic reference while explicitly prohibiting visual copying; defines Board / Floor / Product Page as three renderings of one product identity; extends widget modes with FLOOR_BOOTH; specifies rank-based spatial hierarchy, live founder presence and office hours, interactive booths, physicalized real attention, market-event transitions, live launches, cofounder/hiring/partnership signals, movement/accessibility rules, privacy-safe presence, technical implementation boundaries, success metrics, scope limits and an agent-ready prototype brief. The Floor is optional and must never block or degrade the core Board experience.

39. Commercial comparables and monetization proof

Status: research-backed strategic context, checked August 2026. The purpose is not to claim that OUTRIVL has already found product-market fit. The purpose is to separate the novel combination from the individual commercial primitives, many of which are already monetized at meaningful price points elsewhere.

Core conclusion. No single incumbent located in this research combines a live competitive attention market, persistent interactive product identities, rank-controlled canvas size, playable / interactive listing widgets, optional spatial discovery, and a public promotional pool. However, each underlying economic behavior has strong adjacent precedent: founder/product discovery attracts high-value audiences; vendors pay for prominence; interactive demos and playable ads are real commercial formats; software marketplaces monetize intent data; developer platforms monetize hosting / compute; and launch marketplaces monetize transactions or promotion.

Comparable	Commercial signal	OUTRIVL lesson
Product Hunt	Free product discovery / launches plus paid access to a concentrated founder and early-adopter audience. Current advertising materials state 800k newsletter subscribers, millions of monthly pageviews, self-serve campaigns from $5,000 and managed campaigns with a $10,000/month minimum.	Validates that a founder / product audience itself can support high-value advertising. OUTRIVL should keep basic product presence broadly accessible and monetize guaranteed attention / richer campaign surfaces.
G2	Free product profiles lead into paid seller plans, advertising, buyer-intent and market-intelligence products. Starter is currently advertised at $299/month or $2,999/year for year one; Buyer Intent captures profile, category, comparison and competitor research signals.	Validates the long-term value of first-party software-evaluation behavior. OUTRIVL widget starts, interaction depth, comparisons and continuations could become a richer intent / performance layer if privacy is handled correctly.
BetaList	Current sponsor package is $4,999/month; Boost starts at $99/week, against a claimed founder / early-adopter audience and newsletter.	Shows that even a smaller curated startup-discovery property can sell prominence at meaningful prices.
Uneed	Free launch plus paid launch priority, Pro, newsletter sponsorship, premium placements and monthly advertising.	Validates stacked monetization at indie scale: free supply, optional convenience, subscription and paid visibility can coexist.
AppSumo	No upfront listing fee; AppSumo monetizes through negotiated revenue share. It currently markets a 1.5M entrepreneur community and $113M+ paid directly to partners.	Validates product discovery as a transaction marketplace and the power of concentrated launch demand, even though OUTRIVL should not copy the lifetime-deal model.
Navattic / interactive demos	Navattic reports more than 40,000 interactive demos built in the past year in its 2026 benchmark report, up 43% year over year.	Validates that companies value interactive product demonstration as a standalone growth tool. OUTRIVL makes that capability native to discovery and competition.

Comparable	Commercial signal	OUTRIVL lesson
TikTok Playable Ads	TikTok currently offers global Playable Ads that let users interact with an app preview and exposes creative-level interaction / conversion reporting; assets are constrained and packaged rather than arbitrary webpage code.	Validates the commercial idea that an ad can be an experience. Its constrained playable format is also precedent for OUTRIVL's sandboxed widget runtime.
Hugging Face Spaces	A large directory of runnable AI apps / demos with paid Pro, team / enterprise plans and paid compute / hardware. Hugging Face currently describes 500k+ AI apps in Spaces.	Validates a discovery surface in which the listed object actually runs, plus subscription / hosting economics around that ecosystem.

What this suggests about OUTRIVL revenue architecture

- Layer 1 - free persistent product identity: product page, basic widget / static fallback, searchable metadata and durable history. This maximizes supply and gives every participant something worth sharing even without paid competition.
- Layer 2 - competitive attention: prepaid advertising credits, challenges, premium rank-controlled canvas, sponsored seasons / launch events and eventually publisher-network inventory. This remains the original marketplace revenue engine.
- Layer 3 - OUTRIVL Pro / Cloud: managed AI authoring, widget generation, image creation, richer analytics, brand memory, version history, collaboration, experiments and hosted runtime convenience. BYOK users can avoid managed inference while still paying for higher-value platform features if warranted.
- Layer 4 - intent / performance products: privacy-safe, clearly defined first-party signals such as widget start, meaningful interaction, comparison behavior, product-page continuation and external CTA. Treat this as a later B2B analytics product, not a justification for creepy individual surveillance.
- Layer 5 - distributed interactive inventory: publisher widgets / live King placements, partner revenue share, hosted interactive placements and network-level attribution once OUTRIVL has real demand.

Strategic interpretation. The competitive challenge mechanic does not need to carry the entire company. If paid attacks underperform, OUTRIVL can still have a commercially coherent product as an interactive software-discovery network, widget / demo platform, subscription creative tool or high-intent marketplace. The strongest economic chain to test is: competition creates attention → interactive widgets make that attention unusually measurable and useful → useful intent / performance makes advertisers willing to pay.

Current-source notes for this section

- **Product Hunt advertising:** https://www.producthunt.com/sponsor - 800k newsletter subscribers, millions of monthly pageviews; advertising formats.
- **Product Hunt advertising inquiry:** https://www.producthunt.com/sponsor/inquiry - self-serve from $5,000; managed minimum $10,000/month.
- **G2 plans:** https://sell.g2.com/plans - $299/month or $2,999/year Starter year one; Buyer Intent and Ads add-ons.
- **G2 Buyer Intent:** https://sell.g2.com/quick-start-guides/leverage-insights/learn-about-buyer-intent-signals - profile/category/comparison/competitor behavior as commercial signals.
- **G2 acquisition / network:** https://company.g2.com/news/g2-acquires-capterra-software-advice-getapp - G2 + Capterra + Software Advice + GetApp; 200M+ annual buyer reach cited by G2.
- **BetaList advertising:** https://betalist.com/advertise - $4,999/month sponsorship; Boost from $99/week.
- **Uneed pricing:** https://www.uneed.best/pricing?section=improve - free launch plus Pro, paid priority, sponsorship and ad placements.
- **AppSumo seller page:** https://sell.appsumo.com/ - no upfront listing cost; 1.5M entrepreneur community and $113M+ paid to partners claimed by AppSumo.

- **Navattic 2026 interactive-demo report:** https://www.navattic.com/report/state-of-the-interactive-product-demo-2026 - 40,000+ demos built in the past year; 43% year-over-year increase reported by Navattic.
- **TikTok Playable Ads:** https://ads.tiktok.com/help/article/playable-ads?lang=en - global interactive app-preview ad format and reporting.
- **Hugging Face Pro / Spaces:** https://huggingface.co/pro - 500k+ AI apps described in Spaces; subscription and hosted compute ecosystem.

40. Launch ignition - progressive prize pool, Founding Pro and referrals

Launch thesis. The first public event should be designed as a spectacle with a visible, increasing reward - while preserving the legal separation between free promotional qualification and paid advertising. The prize pool is acquisition spend and social content. It is not the permanent value proposition, and it must never become an uncapped liability.

Progressive launch prize pool

- Announce a guaranteed starting pool that is already funded. The launch should never depend on future sales to honor the minimum promised prize.
- Publish a small number of discrete unlock tiers. Example structure only: GUARANTEED FLOOR → UNLOCK 1 → UNLOCK 2 → FINAL CAP. Exact currency, amounts and thresholds remain open parameters.
- Use a hard maximum announced from the start. Do not use an uncapped formula such as "X% of every participant dollar goes into the pot." A fixed ladder is more legible, financially safer and easier to review legally.
- The trigger may be verified platform volume, qualified marketplace activity, sponsor contributions or a combination. "Volume" must be defined in the official rules and computed from authoritative server-side data. If counsel rejects a revenue-linked trigger, the same visual mechanic can unlock from verified qualified entrants, sponsor top-ups or other non-purchase milestones.
- If revenue is a permitted unlock input, use verified net volume after failed payments, refunds / chargebacks and excluded fraudulent activity rather than optimistic gross client-side counters.
- A participant's paid attacks / advertising must not directly improve their promotional score, eligibility or probability of winning. The free evaluation / skill path described in Section 9 remains the preferred separation.
- Sponsor top-ups can create additional spectacle later. They should be displayed separately from organic platform unlocks so the economics stay interpretable.
- At event close, lock the final unlocked pool and preserve an auditable snapshot of the volume / milestone state that produced it.

Why this is a strong acquisition mechanic. The pool itself becomes a live state variable worth checking and sharing. Every milestone produces content: "$500 UNLOCKED", "$1,000 NEXT", "73% TO NEXT POOL". It gives participants a reason to recruit more attention without requiring OUTRIVL to buy all initial distribution directly.

Founding Pro seeding

- Before broad public launch, recruit a curated Founding Cohort of high-quality products and give them temporary Pro / Cloud access. The objective is not generosity for its own sake; it ensures the marketplace opens with excellent interactive listings rather than empty template cards.
- Prefer time-bounded grants such as 60-180 days rather than lifetime Pro by default. A lifetime giveaway destroys future price discovery and can create permanent service / inference obligations.

- For the earliest 20-50 anchor companies, consider a concierge offer: OUTRIVL helps convert screenshots / product flows into one good interactive widget. This produces golden examples and exposes SDK / authoring friction before thousands of users encounter it.
- Founding participants can receive a visible FOUNDING PRODUCT / SEASON ZERO badge if the badge remains factual and does not imply performance superiority.
- Giveaway Pro access must not change rank, Audience Score, Crown Points or prize qualification. It is tooling / analytics access, not competitive advantage.

Referral rewards

The referral loop should reward contribution to network density without corrupting the market. A referral is only valuable after a new person / company becomes a verified, meaningful participant; raw email signups are too easy to farm.

- Reward examples: Pro days / months, OUTRIVL Cloud generation credits, extra widget-version history, advanced analytics windows, premium share-card customization, faster non-security review lanes, or sponsor-provided software credits.
- Do not reward referrals with rank, Crown Points, Audience Score, promotional score, extra free votes or any hidden boost to the competitive market.
- Define a verified referral event: unique eligible account + verified email / domain as appropriate + completed product / user onboarding + minimum meaningful action. Exact qualification can vary by referral type.
- Use referral IDs at account creation, server-side attribution, device / IP heuristics, domain uniqueness, abuse thresholds and manual review for high-value reward tiers.
- Cap rewards per period and make the economics visible. The system should be generous enough to motivate sharing but not so valuable that professional referral farmers become the primary audience.
- Allow companies to share product-specific referral links and generated launch cards. The best viral unit is the product / market event itself, not a generic "invite friends" modal.

The launch flywheel to test

Free product page + interactive listing → founder shares listing / rank → referral earns useful Pro / Cloud benefits → more quality products enter → the Board and Floor become more interesting → more spectators and interactions → paid advertising becomes more valuable → verified volume may unlock a larger launch pool → the pool creates more social content → more participants arrive.

This is a hypothesis, not a promise of virality. Instrument every transition so OUTRIVL can identify which link actually works rather than attributing all growth to the prize pool.

Promotion safety / legal boundary

- All prize-pool mechanics remain conditional on specialist promotions / gaming counsel and payment-processor approval in the jurisdictions where the event is offered.
- Official rules must define free entry, eligibility, geographic restrictions, event timing, objective scoring, tie-breaks, fraud / disqualification, payout verification, tax handling, pool unlock conditions and the exact relationship - if any - between platform commercial volume and pool size.
- Referral rewards should remain separate from prize qualification. "Refer more people to improve your chance of cash" is not the mechanism being proposed.
- Never advertise an unlocked amount that is not actually funded / reserved. The launch pool is a contractual obligation once publicly promised under final rules.
- The permanent OUTRIVL product should still make sense if promotional cash is switched off entirely.

41. Distribution sequence - Reddit -> X/Twitter -> short-form video -> broader channels

Distribution principle. Launch channels should be sequenced by what OUTRIVL can show convincingly at each stage. Reddit is useful for early founder / builder feedback and concentrated community discussion; X/Twitter becomes stronger once live rank changes and shareable market events exist; TikTok / Reels / Shorts become compelling once the product has motion, interactive widgets and visually dramatic takeovers to film.

Phase A - pre-launch density before promotion

- Do not send a viral post to an empty market. Seed enough high-quality products that a new visitor can immediately browse, interact and understand the hierarchy.
- Curate a small Founding Cohort across Indie / Startup / Open even if early liquidity is uneven. Do not hide the three-class architecture; explain that activity may concentrate differently at first.
- Prepare at least several unusually good widget examples: calculator / configurator, drag-and-drop product miniature, before/after, AI demo with capped runtime, and one playful interaction. These are demonstration assets for distribution as much as product features.
- Have a public rules page, status / FAQ, clear product explanation, share cards, referral links, prize-pool meter if approved, and an email capture / account flow before the first large post.

Phase B - Reddit first

- Post as a builder showing a genuinely new mechanism, not as a generic launch advertisement. The strongest story is: products are not static cards; they run miniature interactive experiences, compete for more canvas, and the launch pool grows through public milestones.
- Use communities where product / startup launches are allowed and respect each community's self-promotion rules. Tailor the post to the subreddit instead of cross-posting identical marketing copy everywhere.
- Lead with one striking GIF / short screen recording or image showing the Board changing rank plus an interactive listing. The concept must be understood before the reader reaches paragraph three.
- Be present in comments. Early objections about fairness, spam, pricing, widget safety and the bounty are product research; capture them as structured launch feedback rather than arguing defensively.
- Track each community with separate referral / attribution parameters. Measure verified product submissions, activated widgets, spectators, return visits and advertiser actions - not merely Reddit clicks.

Phase C - X / Twitter after proof exists

- The product should generate its own social objects: THRONE TAKEN, UPSET, #1 AFTER SPENDING $X, POOL UNLOCKED, FOUNDER ONLINE, LIVE LAUNCH, MOST INTERACTIVE PRODUCT, HIGH CP/$ and similar factual event cards.
- Give every company a share card / clip for its rank, widget and permanent product page. Participant distribution is more credible than the OUTRIVL account repeatedly advertising itself.
- Use short motion clips showing a dethronement: incumbent compresses, challenger expands, widget appears, ticker updates. This communicates the product in seconds.
- Founder / company replies should deep-link into their interactive product page, not only the homepage. The participant should feel that OUTRIVL gives them an asset worth promoting.

Phase D - TikTok, Reels and Shorts when the visual system can carry it

- Short-form video should explain one strange behavior at a time: "This ad is actually usable", "The #1 product gets a bigger interactive canvas", "Watch the homepage physically change when someone takes #1", or "The prize pool just unlocked the next tier."

- Screen recordings are the product. Avoid generic founder talking-head content until there is a reason; the interactive UI, retro-terminal identity, Floor and live market state are already visual hooks.
- Interactive-widget transformations are naturally loopable content: static screenshot → OUTRIVL mini-product; ROW → CARD → FEATURE → THRONE; empty booth → founder goes live; challenger → takeover.
- Use captions and pacing that make sense with sound off. A viewer should understand the mechanism from the first few seconds of motion.
- TikTok itself commercially validates playable ads as a concept, but OUTRIVL distribution videos should promote the marketplace rather than require viewers to understand ad-tech terminology.

Phase E - broader launch channels

- Product Hunt can become a distribution channel rather than only a competitor reference; launching OUTRIVL to the exact audience that uses software launch sites is strategically coherent once the experience is polished.
- Hacker News / Show HN is appropriate only when the implementation and technical story are substantive enough to survive scrutiny; the widget runtime / sandbox / SDK can be part of that story.
- Indie Hackers, maker communities, newsletters, founder Discords / Slacks, design communities and developer communities can be approached with channel-specific angles rather than one universal launch post.
- Later paid acquisition should amplify a proven organic message. Do not buy traffic to discover what the product is supposed to say.

Launch content factory

- Automatically render event cards / short clips from authoritative state changes where possible. A platform that constantly produces shareable market moments can reduce the amount of manual social content the founder must create.
- Create a public "live tape" archive that is indexable / shareable for major events while filtering noise. Each event can link to the product page and current market state.
- Let companies opt into launch reminders and founder office-hour announcements for followers / interested users without building a general social graph on day one.
- Use referral attribution on every generated card / link so OUTRIVL can distinguish platform virality from founder audience size.

42. Sudden-growth readiness and launch operations

Operating assumption. The most dangerous launch failure is not only "nobody comes." A visually novel product with a public prize pool and founder communities can create a short, asymmetric read spike, bot traffic, referral abuse and a burst of writes at exactly the moment public trust matters. OUTRIVL should be able to become temporarily less magical without becoming incorrect.

Non-negotiable reliability hierarchy

1. Money / economic truth: credits, attacks, spending caps, current Throne ownership, promotional unlock state and ledger events must remain transactionally correct or the write path must pause safely.

2. Core public reads: Board snapshots and Product Pages should stay available even if realtime, Floor, AI authoring or community features are degraded.

3. Widget safety and static fallbacks: if custom widget runtime is overloaded / disabled, products still render attractive static states and CTAs.

4. Nice-to-have live features: public presence, particle attention, live viewer counts, animated market tape, comments and expensive AI generation may be throttled or disabled first.

Architecture for a read-heavy viral spike

- Cache public rank / Board snapshots and product metadata aggressively at the edge / application cache layer. The browser does not need a fresh database query for every spectator view.
- Keep the authoritative current state in PostgreSQL but publish compact derived snapshots for reads. Cache invalidation follows successful authoritative transactions; a short delay in spectator display is acceptable, economic inconsistency is not.
- Serve product assets, widget bundles, images and share media from object storage / CDN. Do not proxy large static creative through the application server.
- Use connection pooling and explicit indexes for hot rank, product, event and ledger queries. Avoid unbounded joins in the public homepage request path.
- Realtime Broadcast should fan out semantic events after the transaction commits. Do not make every client poll the database rapidly or write presence movement into PostgreSQL.
- If Realtime becomes unhealthy, fall back to periodic snapshot refresh. A Board that updates every 10-30 seconds is preferable to a live Board that collapses under load.

Protect the transactional path

- All attacks / credit deductions remain one authoritative transaction with row locking / equivalent concurrency control and idempotency. Never resolve simultaneous attacks in the browser.
- Prepaid credits reduce payment-provider calls during a burst. Payment webhooks remain idempotent and separate from the attack transaction.
- If the economic write path is degraded, use a deliberate MARKET PAUSED / CHALLENGES TEMPORARILY PAUSED state. Do not accept ambiguous attacks and reconcile them later.
- Prize-pool unlocks derive from authoritative eligible-volume / milestone records and may lag the UI slightly. Never let a client-side counter unlock cash.
- Preserve an append-only audit trail for pool milestones, challenge acceptance / rejection, credit movement and season changes so launch disputes can be reconstructed.

Queue everything that does not need to block the user

- AI widget generation / image generation / long model calls.
- Share-card / video rendering.
- Email and noncritical notifications.
- Referral qualification and higher-value fraud review.
- Analytics enrichment / aggregation.

- Widget static analysis, preview rendering and non-emergency moderation jobs.
- Publisher / external webhook fanout later.

Feature flags and graceful degradation

Implement independent operational switches before public launch. A single global "site maintenance" toggle is not enough.

- FLOOR_ENABLED - spatial mode can disappear while Board / Product Pages remain.
- PRESENCE_ENABLED - founder / visitor presence off without affecting products.
- REALTIME_ENHANCEMENTS_ENABLED - fall back to snapshot refresh.
- AI_AUTHORING_ENABLED - queue / pause new generations while published widgets continue to run.
- CUSTOM_WIDGET_RUNTIME_ENABLED - fall back to native/static representations if a runtime vulnerability or capacity issue appears.
- REFERRAL_REWARDS_ENABLED - continue attribution but pause reward issuance if fraud spikes.
- PROMO_POOL_PUBLIC_ENABLED - hide / pause the promotional event only according to official rules and counsel-approved contingency language; never silently alter promised terms.
- CHALLENGES_ENABLED - economic kill switch; when false, reads continue and the UI clearly reports that attacks are paused.

AI / Cloud spend protection

- BYOK is naturally isolated from OUTRIVL model spend, but managed Cloud must have per-account and global quotas from day one.
- Set hard daily / monthly generation budgets and a global emergency circuit breaker. Viral signup volume must not create an unbounded OpenAI bill.
- Author-time generation is asynchronous and can queue. Runtime-AI widgets require explicit per-widget / per-campaign budgets and should degrade with a clear "limit reached" state rather than silently burning platform funds.
- Cache deterministic / reusable outputs where appropriate and avoid regenerating responsive variants on every page view.

Referral / bot / moderation surge controls

- Expect the prize pool and Pro rewards to attract abuse immediately. Rate-limit account creation, referral claims, widget events and challenge endpoints independently.
- Use Turnstile / risk checks on sensitive actions, email / domain verification where appropriate, duplicate-account heuristics and manual review for suspicious high-value rewards.
- Do not expose exact anti-fraud thresholds publicly. Keep all reward grants reversible until the qualifying event passes fraud checks.
- Build an admin console capable of suspending a product, pausing a widget version, invalidating a referral chain, freezing a credit account, rejecting a challenge and annotating the audit reason.
- Prepare moderation copy and escalation rules before launch; deciding what constitutes a malicious interactive widget during a viral incident is too late.

Capacity and launch rehearsal

- Load-test the read path separately from the challenge / write path. The likely public spike is many more spectators than advertisers.
- Test at least one order of magnitude above the expected launch audience, then run a sharper synthetic read burst to reveal connection / cache / asset bottlenecks. Exact targets should be based on the deployed environment rather than copied from this document.
- Run concurrency tests that deliberately submit simultaneous attacks against the same Throne and confirm exactly one valid result / deterministic ordering under the chosen transaction rules.
- Test the site with Realtime disabled, AI disabled and custom widgets disabled. Graceful-degradation modes are only real if they have been exercised.

- Rehearse payment webhook duplication, delayed webhook delivery, refund / chargeback adjustment, referral fraud and a promotional milestone reversal before the public event.
- Keep database backups / point-in-time recovery, object-store versioning where practical, and all widget / application code in version control. A viral launch is not the time to discover that the only copy of a production asset exists on one machine.

Launch-day operational dashboard

- Traffic: requests, cache hit rate, p95 / p99 latency, error rate, bandwidth and hot routes.
- Database: connection pressure, slow queries, lock waits, transaction failures, replica / service health if used.
- Economic: accepted / rejected attacks, credit balance anomalies, payment webhook failures, pool unlock state and any reconciliation exceptions.
- Abuse: signup velocity, Turnstile / risk failures, duplicate referrals, suspicious widget events and moderation queue size.
- AI: managed generation queue depth, model errors, daily spend and per-account budget exhaustions.
- Product: activated listings, widget-start rate, product-page continuation, referrals that become verified participants, first paid advertiser actions and re-attack rate.
- A single founder should be able to see these health signals without querying production manually. Use Sentry / platform logs / Supabase metrics / PostHog / payment dashboards as appropriate; avoid building a custom observability company before launch.

If it actually takes off

Do not respond to virality by rewriting the architecture. First buy headroom: raise managed database / hosting limits, increase cache capacity, move expensive jobs behind queues, tighten abusive endpoints, and turn off nonessential realtime / Floor effects if necessary. Preserve the public experience and economic correctness. The existing Next.js + Supabase/Postgres + Upstash + Vercel shape is intentionally chosen so early scale can be purchased before it must be re-architected.

Waitlist / queue fallback. If submission / moderation capacity, not read traffic, becomes the bottleneck, keep the marketplace public but place new product publishing into a clear review queue. A visitor spike should not force OUTRIVL to publish unsafe or low-quality widgets just to keep up.

43. v0.7 change note

v0.7 adds a research-backed commercial-comparables section and reframes OUTRIVL as a layered business: free interactive identity, competitive attention, Pro / Cloud, privacy-safe intent / performance products and later distributed interactive inventory. It records current Product Hunt, G2, BetaList, Uneed, AppSumo, Navattic, TikTok Playable Ads and Hugging Face Spaces evidence as adjacent validation rather than proof of OUTRIVL product-market fit.

It also formalizes the launch strategy: a guaranteed and capped progressive prize pool with discrete unlock tiers; Founding Pro seeding and concierge widget creation for an anchor cohort; verified referral rewards that never alter ranking or prize qualification; a distribution sequence beginning with Reddit, then X/Twitter, then short-form video and broader founder / developer channels; and a launch-day growth loop built around shareable market events.

Finally, v0.7 adds a sudden-growth operating plan: read-heavy caching, transactional write protection, queues, feature flags, graceful degradation, AI spend caps, anti-fraud controls, load / concurrency rehearsal, a launch dashboard, safe pause states and a waitlist fallback. The governing principle is that OUTRIVL may temporarily become less magical under pressure, but it must not become economically incorrect.

OUTRIVL WORKING PRODUCT BRIEF - v0.7 - commercial proof, launch ignition and surge readiness

44. Exact Claude Design -> Codex -> Claude Code handoff sequence

Why this section exists. Earlier versions define each tool role and the build order, but the operational handoff should be even more explicit so the founder does not have to reconstruct the intended loop from several paragraphs. This is the default first-cycle sequence. It is deliberately asymmetric: Claude Design owns art direction, Codex owns visual implementation, and Claude Code owns production engineering. None of the three should independently reinterpret the whole product.

Base three-tool sequence: Claude Design → Codex → Claude Design → Codex → visual lock → Claude Code → Codex visual QA → Claude Code engineering QA. **Harness-enhanced sequence:** Claude Design → Lavish plan/review where useful → Codex → Claude Design → Codex → visual lock → Claude Code → Codex visual QA → Claude Code / No Mistakes engineering QA.

1. **Claude Design - visual implementation guide.** Give Claude Design the current OUTRIVL brief, the approved Board image, approved Product Page image, and Founder Floor mechanic reference. Ask it to sharpen the existing direction, not invent a replacement brand. The output should specify typography direction, grid / proportion logic, the correct amount of nostalgic pixel treatment, borders, texture / dither rules, palette behavior, data density, motion grammar, interactive-widget presentation, and the visual anti-patterns that would make OUTRIVL collapse into generic SaaS.
2. **Lavish - make the plan inspectable** when a scene or engineering plan benefits from visual review. Convert the proposed Board composition, Product Page state map, widget runtime diagram, migration plan, or other complex plan into an interactive HTML artifact. The founder reviews the artifact visually and leaves inline feedback before expensive implementation. Lavish is a review/planning surface, not the final design system or production runtime.
3. **Codex - coded visual prototype.** Codex receives the brief, Claude Design output, approved reference pack, and any accepted Lavish comments. It builds the scene in the browser with hardcoded fixtures first. For the initial Board, do not let backend setup, auth, payments, database abstraction, or a generic component library slow the visual proof. Use browser automation and fixed screenshots to iterate toward the golden references.
4. **Claude Design - external visual critique.** Feed the actual Codex screenshots back to Claude Design. It should identify where implementation drifted: too much / too little pixel treatment, weak rank hierarchy, generic card geometry, insufficient Bloomberg-style density, poor typography, widget feeling bolted on, or motion that feels decorative rather than stateful. Claude Design critiques; it does not rewrite the architecture.
5. **Codex - convergence pass.** Codex applies the critique, reruns the page, captures the same fixed breakpoints, and repeats until the founder explicitly approves the scene. Once approved, store the screenshots under /references/golden/ and treat them as visual regression contracts.
6. **Repeat the visual loop across the core scenes.** Board / Throne first, Product Page second, Dethronement third, Widget Interaction fourth, Advertiser Widget Studio fifth, and Spatial Floor only after the core market surfaces are convincing. The exact scene list may evolve, but the rule remains: prove visual behavior before productionizing it.
7. **Visual lock.** The founder explicitly promotes a coded state to golden. Before this point, the design may move aggressively. After this point, engineering refactors are not permission to simplify or normalize the visual output.
8. **Claude Code - productionization.** Claude Code takes the approved prototype and makes it maintainable and real: domain types, component boundaries, Supabase/Postgres, transactional attack / credit logic, Realtime, Redis only for non-authoritative fast state, PostHog/Sentry, widget sandbox, SDK packaging, BYOK / OUTRIVL Cloud boundaries, tests, deployment, and operational controls. Its instruction is preserve rendered behavior while improving structure.

9. **Codex - visual regression QA after productionization.** Compare production screenshots against golden screenshots at the same breakpoints. Fix hierarchy, spacing, typography, motion, widget presentation and other visual regressions without casually changing economic or security logic.
10. **Claude Code plus No Mistakes - engineering gate.** Run tests, linting, type checks, transactional / concurrency tests, security checks, widget-sandbox verification, degraded-mode tests and PR review in fresh context. Any change to a golden scene is a visual regression unless the founder explicitly accepts a new golden state.

How to use the loop after the design system stabilizes

Not every later feature needs the full nine-stage cycle. Once OUTRIVL has a mature visual grammar, the default feature loop can compress to: brief / accepted plan → Codex visual implementation → Claude Code productionization → Codex visual regression → engineering gate. Return to Claude Design when a feature creates a new visual primitive, new interaction mode, or unresolved art-direction problem.

- Do not ask all agents to "build OUTRIVL" simultaneously. Give each agent a bounded role and an explicit artifact to hand to the next stage.
- Do not let Claude Code redesign while refactoring. If a production constraint makes the golden design impossible, surface the constraint as a decision rather than silently simplifying it.
- Do not let Codex alter money / ranking truth in order to make a visual prototype convenient. During prototype stages it may use fake state; once connected to production, authoritative state rules from the brief win.
- Do not send unresolved founder judgment calls into autonomous overnight loops. Brand, pricing, promotional rules, irreversible schema decisions, security permissions and economic semantics require explicit human approval.
- Prefer small reversible commits and clear scene-level acceptance criteria so work can be compared, reverted and parallelized safely.

45. Agent harness layer - AXI, Lavish, Treehouse, gnhf, Firstmate and No Mistakes

Inspiration and scope. Kun Chen, a former L8 principal engineer at Meta and Microsoft, has published an agentic-engineering workflow organized around plan → implement → validate, plus a family of open-source tools for agent-ergonomic interfaces, visual plan review, parallel worktrees, long-running measurable loops and validation gates. OUTRIVL can adopt these operating ideas without making any third-party tool a hard architectural dependency. The value is the flow: improve planning quality, make agent work inspectable, isolate parallel changes, and validate in fresh context before merge.

Important terminology. Lavish is primarily a visual HTML planning / review interface; AXI is a set of design principles and reference implementations for agent-ergonomic command-line interfaces; Treehouse / gnhf / Firstmate are orchestration or work-management tools at different levels; No Mistakes is a validation / PR gate. Treating them as one monolithic "agent harness" hides the useful separation of responsibilities.

AXI - make tools legible to agents

- Use AXI-style interfaces where they measurably reduce tool-call overhead and ambiguity. The official AXI catalog currently includes gh-axi for GitHub operations, chrome-devtools-axi for browser automation, lavish-axi for human review, and quota-axi for local model quota / usage visibility.
- For OUTRIVL visual iteration, chrome-devtools-axi is particularly relevant: Codex / Claude Code can navigate the running app, manipulate controls, extract state and support screenshot / browser validation without relying on a large undifferentiated browser-tool schema.
- For repo operations, gh-axi can be preferred when the current agent environment supports it. The goal is not allegiance to one CLI; it is compact, deterministic, discoverable operations with enough context for the agent to choose the next action.
- Keep the normal GitHub CLI / browser tooling available as a fallback. Third-party agent tooling must never become the only way a maintainer can operate the repository.

Lavish - visual planning before expensive implementation

- Install / use Lavish in agent environments that support its skill / CLI workflow, particularly Claude Code. Use it for plans that are easier to judge spatially than as Markdown: UI composition, state machines, architecture maps, widget capability matrices, migration plans, launch control panels and review reports.
- The preferred human loop is: agent generates an HTML artifact → founder reviews in browser → founder annotates specific parts → feedback returns to the agent → plan updates → implementation begins only when the plan is accepted.
- For visual work, Lavish complements Claude Design rather than replacing it. Claude Design decides art direction; Lavish makes the proposed implementation plan and tradeoffs inspectable; Codex then makes the real browser experience.
- Store accepted decisions back into durable repo files. An interactive planning artifact is useful during review, but the repository brief / ADR / design constitution remains the long-lived source of truth.

Treehouse - safe parallel worktrees

- Use isolated worktrees when two or more agents can work on genuinely independent tasks. Examples: one agent builds Product Page behavior while another writes widget-runtime tests; one investigates load testing while another works on share-card generation.
- Do not parallelize tightly coupled edits to the same visual surface simply because multiple agents are available. Parallelism that creates merge conflict or competing design interpretations is negative throughput.
- Each parallel task should have a small contract: objective, allowed files / subsystem, acceptance checks, prohibited changes, and expected handoff artifact.

- Rebase / refresh from the current golden mainline before a task starts and gate each branch independently before merge.

Firstmate - one liaison for a visible crew

- Consider Firstmate once OUTRIVL has enough parallel engineering work that the founder is spending more time coordinating agent sessions than making product decisions. Its useful pattern is one liaison agent that dispatches bounded tasks to isolated crewmates and escalates only real decisions.
- Use it for implementation / investigation throughput, not for outsourcing product taste. The founder remains the captain for art direction, economics, legal / promotional choices, security permissions and irreversible architecture decisions.
- Scout tasks are useful for uncertain questions that should return a report rather than modify code: performance investigation, library comparison, CSP research, payment-provider constraints, browser compatibility or an SDK design review.

gnhf - bounded autonomous progress, not autonomous product management

- Use long-running loops only for objectives with measurable progress and safe rollback. Good candidates: increase test coverage in a defined module, eliminate a known class of TypeScript errors, improve Lighthouse / bundle targets without visual regressions, add missing widget-runtime fixtures, reduce failing tests, or migrate a bounded API surface.
- Each iteration should produce one small committed change when successful and discard / roll back failed attempts. Set runtime, token and iteration caps.
- Do not send vague objectives such as "make OUTRIVL better", "redesign the Board", "optimize monetization", or "improve security" into an unattended loop. Those objectives contain judgment calls that can create large silent drift.
- Never give an unattended loop unrestricted production credentials, payment operations, prize-pool controls or broad secret access. Use least privilege and a non-production environment by default.

No Mistakes - validation as a separate phase

- Use a fresh-context validation gate after meaningful agent-written changes. The validator should reproduce the task, inspect the diff, run automated checks, identify missing tests, perform targeted manual / browser validation where appropriate, and distinguish safe automatic fixes from founder judgment calls.
- For OUTRIVL, supplement generic code review with project-specific evidence: golden screenshot diffs, transactional attack concurrency tests, credit-ledger invariants, widget CSP / sandbox tests, BYOK secret-isolation tests, graceful-degradation checks and accessibility / performance budgets.
- Do not accept "all tests passed" as sufficient evidence for a visual or economic change. Visual state and money state require their own dedicated evidence.
- Keep merge authority human-controlled at first. Automation may prepare clean PRs and fix low-risk findings, but security / economics / promotional-rule changes should escalate.

Recommended OUTRIVL harness profile by phase

- Visual discovery: Claude Design + Lavish where useful + Codex + chrome-devtools-axi / Playwright + founder approval. Low parallelism; maximize taste and iteration quality.
- Productionization: Claude Code as primary engineer, Treehouse for isolated independent work, Codex for visual-regression passes, gh-axi for repository operations where useful, and No Mistakes as the merge gate.
- High-throughput mature repo: Firstmate may coordinate multiple bounded crewmates; gnhf may run measurable maintenance / test / performance loops; golden visual / economic / security invariants remain hard gates.
- Launch / surge period: reduce autonomy rather than increase it. Freeze risky dependencies, require small PRs, keep the economic write path conservative, and avoid unattended refactors while live money / promotional state is active.

Repository files to add for the harness

- /AGENTS.md - concise agent operating contract: read order, invariant list, prohibited autonomous changes, commands and evidence expectations.
- /docs/design-constitution.md - stable visual grammar distilled from the brief and golden screenshots.
- /docs/agent-workflow.md - the exact Section 44 handoff loop plus when to use Lavish, parallel worktrees, long-running loops and validation gates.
- /docs/validation-contract.md - required tests / screenshots / invariants by subsystem before a PR can merge.
- /docs/decisions/ - ADR-style records for architecture, economics, security and visual-system changes.
- /references/golden/ - fixed screenshots by scene / breakpoint and, later, short motion recordings for signature transitions.
- /.claude/skills/ or equivalent project skill directory - only reviewed / pinned skills needed by the active harness. Do not indiscriminately install every available agent tool.

Third-party-tool safety and portability

- These tools are fast-moving open-source projects. Pin versions / commits where practical, review install scripts and permissions, and upgrade deliberately rather than pulling latest into a production-critical workflow without review.
- Keep OUTRIVL build / test / deploy commands conventional and documented so the project remains operable without AXI, Lavish, Treehouse, gnhf, Firstmate or No Mistakes.
- Do not expose production secrets merely to improve agent convenience. Prefer scoped development credentials, dedicated project keys, local mocks and explicit approval for sensitive actions.
- Agent traces, plans and validation reports can contain proprietary source / configuration context. Store them according to the confidentiality expectations of the repo and avoid publishing them accidentally in public artifacts.

Current-source notes for this section

- **AXI - Agent eXperience Interface:** https://axi.md/ - agent-ergonomic CLI principles; official catalog includes gh-axi, chrome-devtools-axi, lavish-axi and quota-axi.
- **Kun Chen AXI repository:** https://github.com/kunchenguid/axi - open-source AXI principles and benchmark material.
- **Lavish:** https://github.com/kunchenguid/lavish-axi - HTML planning / annotation and human-review workflow; skill / CLI integration for coding agents.
- **Treehouse:** https://github.com/kunchenguid/treehouse - reusable isolated Git worktree management for parallel agent tasks.
- **gnhf:** https://github.com/kunchenguid/gnhf - long-running agent orchestrator with small committed iterations, failure rollback and caps.
- **Firstmate:** https://github.com/kunchenguid/firstmate - one liaison / crew pattern using isolated worktrees and ship / scout tasks.
- **No Mistakes:** https://github.com/kunchenguid/no-mistakes - local validation gate that runs an AI-driven review / test pipeline before forwarding a clean branch / PR.
- **Behind the Craft interview with Kun Chen:** https://podcasts.apple.com/us/podcast/how-this-ex-meta-l8-engineer-ships-40-prs-a-day-with/id1736359687?i=1000771546797 - public discussion of plan → code → validate, Lavish, parallel agents and No Mistakes.

46. v0.8 change note

v0.8 makes the human / agent operating sequence explicit rather than implicit. It records the canonical first-cycle handoff as Claude Design → Lavish planning / review where useful → Codex visual prototype → Claude Design critique → Codex convergence → visual lock → Claude Code productionization → Codex visual regression → Claude Code / No Mistakes engineering validation. This supplements, rather than replaces, the role boundaries and build sequence already defined in Section 35.

It also adds an optional agent-harness layer inspired by Kun Chen's published workflow and open-source tooling: AXI-style agent ergonomics for GitHub / browser operations, Lavish for visual plan review, Treehouse for isolated parallel worktrees, Firstmate for one-liaison multi-agent coordination, gnhf for bounded measurable autonomous loops, and No Mistakes for fresh-context validation / PR gating. These remain workflow accelerators, not hard runtime dependencies of OUTRIVL, and the brief explicitly requires portability, version pinning / review, least-privilege credentials and human control over high-judgment or high-risk decisions.

OUTRIVL WORKING PRODUCT BRIEF - v0.8 - exact agent handoff and agent-harness operating model